

B.M.S. COLLEGE OF ENGINEERING BENGALURU
Autonomous Institute, Affiliated to VTU



Lab Record

Software Engineering and Object-Oriented Modeling

Submitted in partial fulfillment for the 5th Semester Laboratory

Bachelor of Engineering
in
Computer Science and Engineering

Submitted by:

PADMASHREE JAIN D
1BM23CS223

Department of Computer Science and Engineering
B.M.S. College of Engineering
Bull Temple Road, Basavanagudi, Bangalore 560 019
August 2025-December 2025

B.M.S. COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING



CERTIFICATE

This is to certify that the Object-Oriented Analysis and Design(22CS6PCSEO)
laboratory has been carried out by **Padmashree Jain D(1BM23CS223)** during the
5th Semester August 2025-December 2025

Signature of the Faculty Incharge:

Name of Your Batch Incharge and designation: Dr. Adarsha Sagar H V

(Assistant Professor)

Department of Computer Science and Engineering
B.M.S. College of Engineering, Bangalore

Table of Contents

1. Hotel Management System
2. Credit Card Processing
3. Library Management System
4. Stock Maintenance System
5. Passport Automation System

1. Hotel Management System

Problem Statement
SRS-Software Requirements Specification
Class Diagram Requirements: Minimum 7 classes, 4 attributes and 4 operations in each class, associations, association name, association end names, multiplicity, association class, enumeration, qualified association, aggregation, composition, generalization, ordered, sequencing, multiplicity of attribute, brief description of the Class diagram
State Diagram: one simple state diagram, one advance state diagram (either Sub Machine or Nested State, include any one of the concurrency in your design) Requirements: minimum of 6 states for both the state diagram with state name, do activities, events, guard condition, brief description of State diagram
Use-Case Diagram: Minimum 5 use cases to be identified, and design one simple use case diagram and one advanced use case diagram(ie using include,extend and generalization relationship) and explanation as there in the solution manual.
Sequence Diagram: Write the scenario for any two use case transaction completely. and design the simple sequence diagram with minimum of 5 objects and communication between them. Design advanced sequence diagram using passive and transient objects. Brief explanation for each
Activity Diagram: Design the simple activity diagram showing the algorithm or workflow. Design the advanced activity diagram with swimlanes showing the responsible person for swimlane and also write the brief explanation for both.

1.1Problem Statement

The Hotel Management System (HMS) is designed to computerize and streamline core hotel operations such as room booking, check-in/check-out, housekeeping, billing, payment processing, and reservation management.

The system should eliminate manual errors, improve efficiency, maintain accurate real-time

room availability, and enhance the overall guest experience.

This system helps hotel staff manage guest details, room status, payments, and housekeeping schedules in a centralized, user-friendly manner.

1.2 SRS-Software Requirements Specification

1. Hotel Management System

The design & implementation of HMS is time consuming, inefficient, error prone. Tasks such as room booking, check-in, check-out, billing, staff scheduling & reporting are often done on paper (or) with basic tools, which can lead to double booking, billing mistakes, loss of records, & poor customer service. There is a need for a computerized Hotel Management System that can handle all these tasks in one place.

SRS:

1. Introduction

1.1 Purpose of the Document

The purpose of this document is to define the functional & non-functional requirements for the Hotel Management System (HMS). The system should make it easier to manage reservations, track room availability, generate accurate bills, manage staff records, & provide timely reports, while also improving customer experience & reducing workload for hotel staff.

1.2 Scope of the Document

The HMS is a web/desktop-based application that enables hotels to manage:

- Customer reservations & cancellations
- Room allocation & status tracking
- Billing & invoice generation
- Staff duty schedules & payroll
- Restaurant/food ordering & integration with room service
- Generating daily, weekly & monthly business reports

The system will:

- Reduce manual errors in booking & billing.
- Improve customer experience through faster service.
- Enable hotel administrators to monitor performance in real-time.
- Support both small hotels & large chains with scalability.

1.3 Overview

The system will consist of 3 major modules.

1. Customer Module - handles bookings, cancellations & payments
 2. Staff Module - manages employee records, schedules & tasks
 3. Administration Module - manages hotel operations, financial reporting & analytics
- The system provides a secure login for different roles: Admin, Receptionist, Customer & Manager.

2) General Description

- Users of the system:
 - Hotel Administrators/managers
 - Receptionists
 - Customers
 - Support staff (cleaning, maintenance)
- Assumptions & Dependencies:
 - The system requires internet connectivity for real-time updates
 - Database must support multi-tenancy
 - Payment gateway integration is required for online payments

3) Functional Requirements

1. User Authentication:
 - Admin, staff & customer must login securely
 - Admin roles should have different access
 - Staff roles should have different access
2. Room Booking & Cancellation:
 - Customer can check room availability
 - Customer can book rooms manually
 - Receptionist can allocate rooms manually
 - System should automatically update availability after bookings/cancellations

2. Check-In / Check-Out Process

1. Receive customer details at check-in
2. Update room status upon check-out
3. Billing & Payments:
 - Generate automated invoices for customers
 - Support multiple payment modes (cash, card, online)
 - Maintain payment history
4. Staff Management:
 - Store staff details (personal, employment, payroll)
 - Manage room shifts & leave requests
5. Restaurant / Food Service:
 - Allow room service orders to be linked to the customer's bill
 - Maintain menu & update stock levels

Reporting

- Generate occupancy reports, revenue reports
- Performance reports
- Support export to PDF/Excel

UI/UX Requirements

- User interface should be accessible on desktop, tablet, mobile
- Responsive design with real-time updates
- Simple navigation with clear, bold charts
- Graphical reports with charts
- Hardware Interface:
 - Works on hotel computers with min. specs: Intel i5, 8GB RAM, Storage: 500GB HDD
 - Support barcode / QR scanner for check-in/out
- Software Interface:
 - OS: Windows / Linux
 - Database: MySQL / PostgreSQL
 - Web Frameworks: Django / Java Spring / Node.js
 - Web Frameworks: Django / Java Spring / Node.js
 - Payment Gateway: Razorpay / PayPal / Stripe

4) Performance Requirements

- The system should handle at least 200 concurrent users
- Response time for room availability check should be < 2 seconds
- Payment transactions should process within 5s
- Database should support up to 1 million records without performance degradation

5) Design Constraints

- System must comply with GDPR / Indian IT Act for data privacy
- Only authorized users can access sensitive data
- The UI should follow accessibility standards (WCAG)
- System should integrate with third-party APIs (Payment, Email, SMS)

6) Non-functional Attributes

1. Security:
 - Data encryption using AES-256 for customer details
 - Secure HTTPS communication
 - Role-based access control (RBAC)
2. Reliability:
 - 99.5% uptime requirement
 - Daily automated database backup
3. Scalability:
 - Ability to expand from single hotel to multi-chain management
4. Maintainability:
 - Modular design for easy updates
 - Comprehensive documentation for dev & non-tech
5. Usability:
 - Easy to use interface for non-tech hotel staff
 - Multilingual support

7) Total estimated budget for HCM is ₹15,00,000.

Phase	Task	Duration
Phase 1	RA	2 weeks
Phase 2	System Design	3 weeks
Phase 3	Development	8 weeks
Phase 4	Testing & QA	4 weeks
Phase 5	Deployment & Training	1 week

1.3 Class Diagram

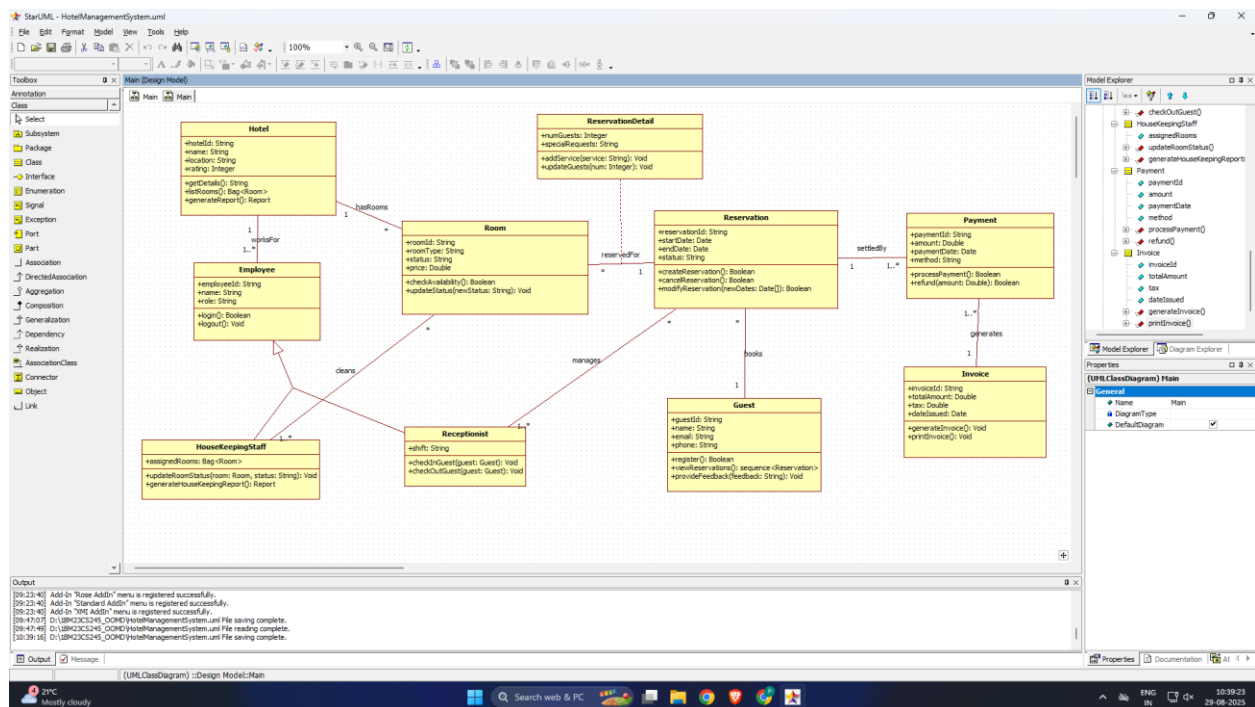


Fig 1.1

The class diagram for the Hotel Management System models the primary entities involved in hotel operations and their relationships.

It includes classes such as **Hotel**, **Room**, **Employee**, **Receptionist**, **HouseKeepingStaff**, **Guest**, **Reservation**, **ReservationDetail**, **Payment**, and **Invoice**, along with the enumeration **RoomStatus** and **Status**.

2.1 Major Classes & Their Responsibilities

1. Hotel

- **Attributes:** hotelId, name, location, rating
- **Operations:** getDetails(), listRooms(), generateReport()
- Represents the hotel entity and provides high-level operations for retrieving details and generating reports.

2. Room

- **Attributes:** roomId, roomType, status (RoomStatus), price
- **Operations:** checkAvailability(), updateStatus()
- A central entity that stores room information and tracks availability.

3. Guest

- **Attributes:** guestId, name, email, phone
- **Operations:** register(), viewReservations(), provideFeedback()
- Represents the customer who makes reservations and stays at the hotel.

4. Employee (Parent Class)

- **Attributes:** employeeId, name, role
- **Operations:** login(), logout()
- Superclass for hotel staff.

5. Receptionist (Subclass of Employee)

- **Attributes:** shift
- **Operations:** checkInGuest(), checkOutGuest()
- Handles front-desk operations.

6. HouseKeepingStaff (Subclass of Employee)

- **Attributes:** assignedRooms
- **Operations:** updateRoomStatus(), generateHouseKeepingReport()
- Maintains room cleanliness and updates room condition.

7. Reservation

- **Attributes:** reservationId, startDate, endDate, status
- **Operations:** createReservation(), cancelReservation(), modifyReservation()
- Represents the booking made by a guest.

8. ReservationDetail

- **Attributes:** numGuests, specialRequests
- **Operations:** addService(), updateGuests()
- Stores additional guest requirements linked to a reservation.

9. Payment

- **Attributes:** paymentId, amount, paymentDate, method, paymentStatus
- **Operations:** processPayment(), refund()

- Tracks billing and payment settlement.

10. Invoice

- **Attributes:** invoiceId, totalAmount, tax, dateIssued
- **Operations:** generateInvoice(), printInvoice()
- Generates the final invoice for completed stays.

11. RoomStatus (Enumeration)

- Available
- Occupied
- Reserved
- OutOfService

12. Status (Enumeration for Payment)

- Pending
- Completed
- Refunded

2.2 Associations & Multiplicity Explanation

Hotel — Room

- **1 to *** (one hotel has many rooms)
- Represents that a hotel manages multiple rooms.

Employee — Hotel

- **1 to *** (hotel employs multiple employees)

Receptionist — Reservation

- Receptionist **manages** reservations (1 to *)

Guest — Reservation

- **1 guest can make many reservations**
- Multiplicity: *1.. on reservation side, 1 on guest side**

Room — Reservation

- A reservation is **for one room**, but a room can be reserved **multiple times** over time.

Reservation — ReservationDetail

- **1 to 1 association (composition)** → reservation detail cannot exist without the reservation.

Reservation — Payment

- One reservation can have **one or many payments** (installments/partial payments in advanced cases).

Payment — Invoice

- Payment **generates** an invoice ($1..* \rightarrow 1$).

HouseKeepingStaff — Room

- One staff member can **clean multiple rooms**.
- Relationship: **1 to * (cleaning)**

2.3 Aggregation, Composition & Special Cases

Composition:

- **Reservation → ReservationDetail**
ReservationDetail cannot exist without Reservation.

Aggregation:

- **HouseKeepingStaff → assignedRooms**
Rooms are “assigned” but not owned by staff.

Inheritance:

- **Employee** superclass with subclasses **Receptionist** and **HouseKeepingStaff**

Qualified Association:

- Room search is often **qualified by roomId or roomType**.

Ordered Association:

- Guest’s reservation list is normally **ordered by date**.

1.4 Advanced Class Diagram

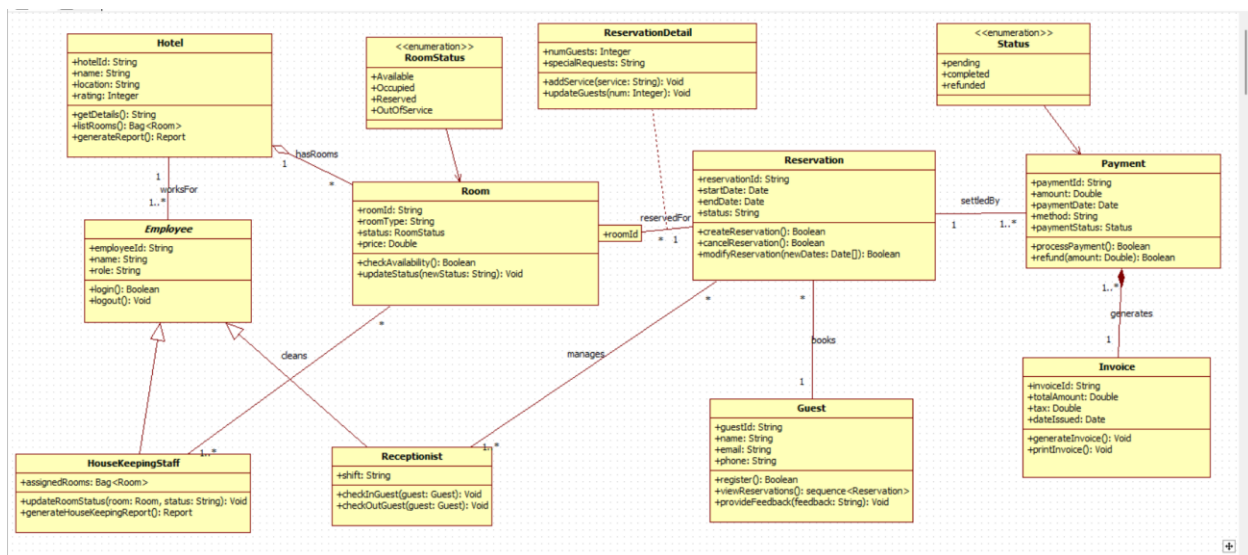


Fig 1.2

1.5 State Diagram

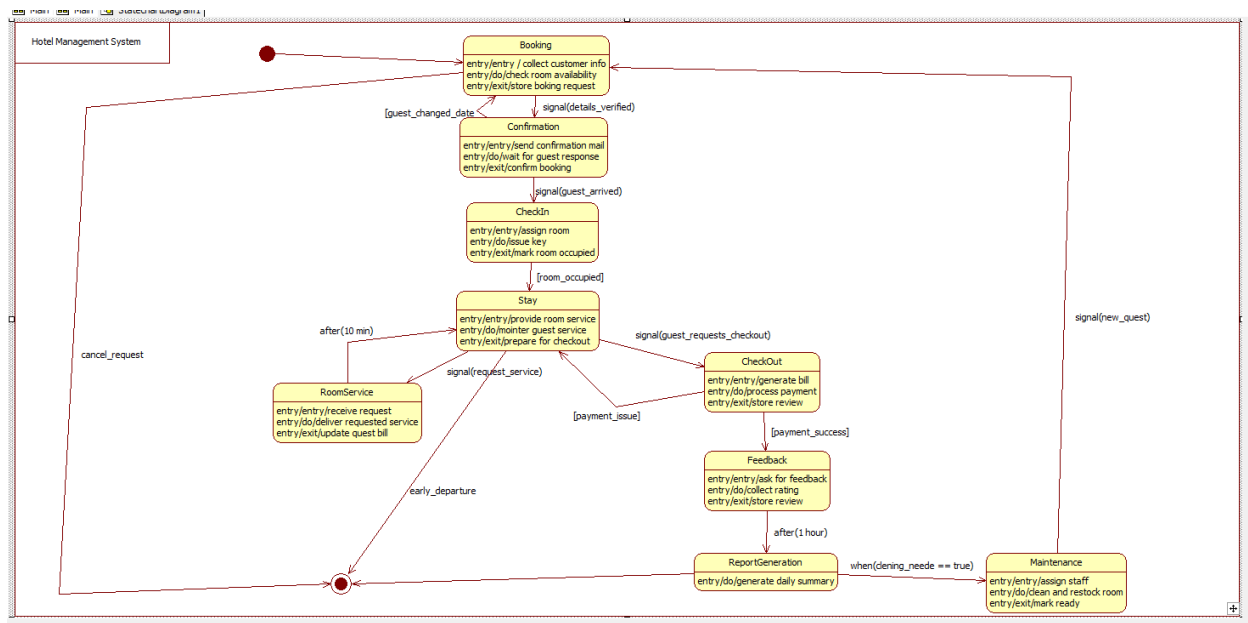


Fig 1.3

States:

1. Available
2. Reserved
3. Occupied
4. Under Cleaning

5. Out Of Service

Example transitions:

- *Available* → *Reserved* (on reservation creation)
- *Reserved* → *Occupied* (on guest check-in)
- *Occupied* → *Under Cleaning* (after check-out)
- *Under Cleaning* → *Available* (after housekeeping update)

1.6 Advanced State Diagram

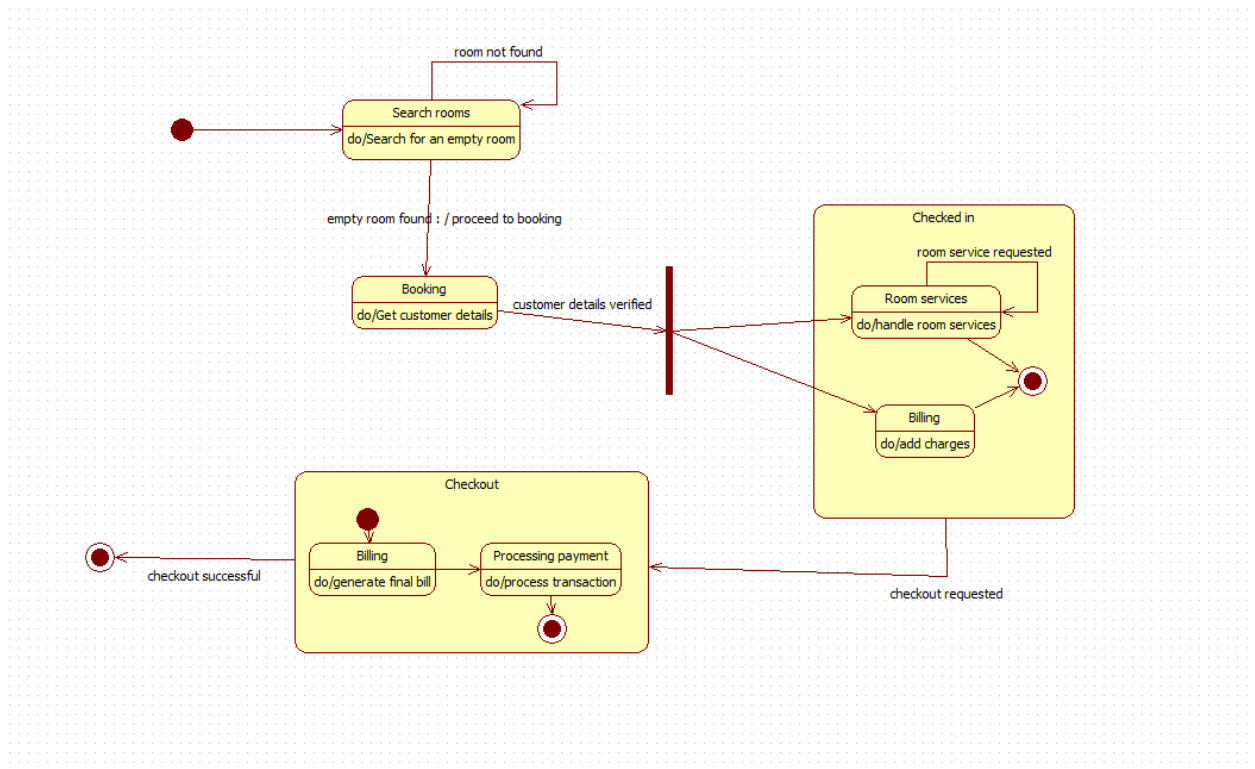


Fig 1.4

1.7 Use Case Diagram

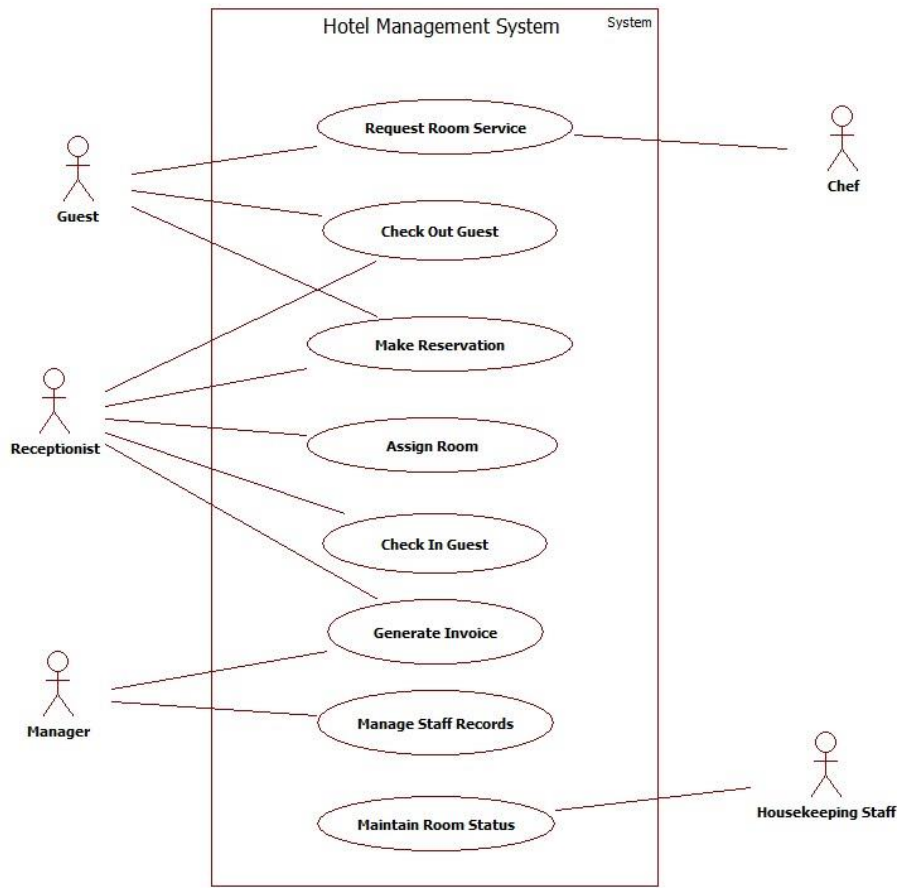


Fig 1.5

The simple use case diagram represents the basic hotel operations performed by the main users. The **Guest** can make reservations, request room service, and check out. The **Receptionist** handles back-end tasks such as check-in, check-out, assigning rooms, and generating invoices. The **Manager** manages staff-related operations, while **Housekeeping Staff** maintain room status. A **Chef** supports room service requests.

This diagram captures the core interactions between users and the system without showing detailed relationships like include/extend.

1.8 Advanced Use Case Diagram

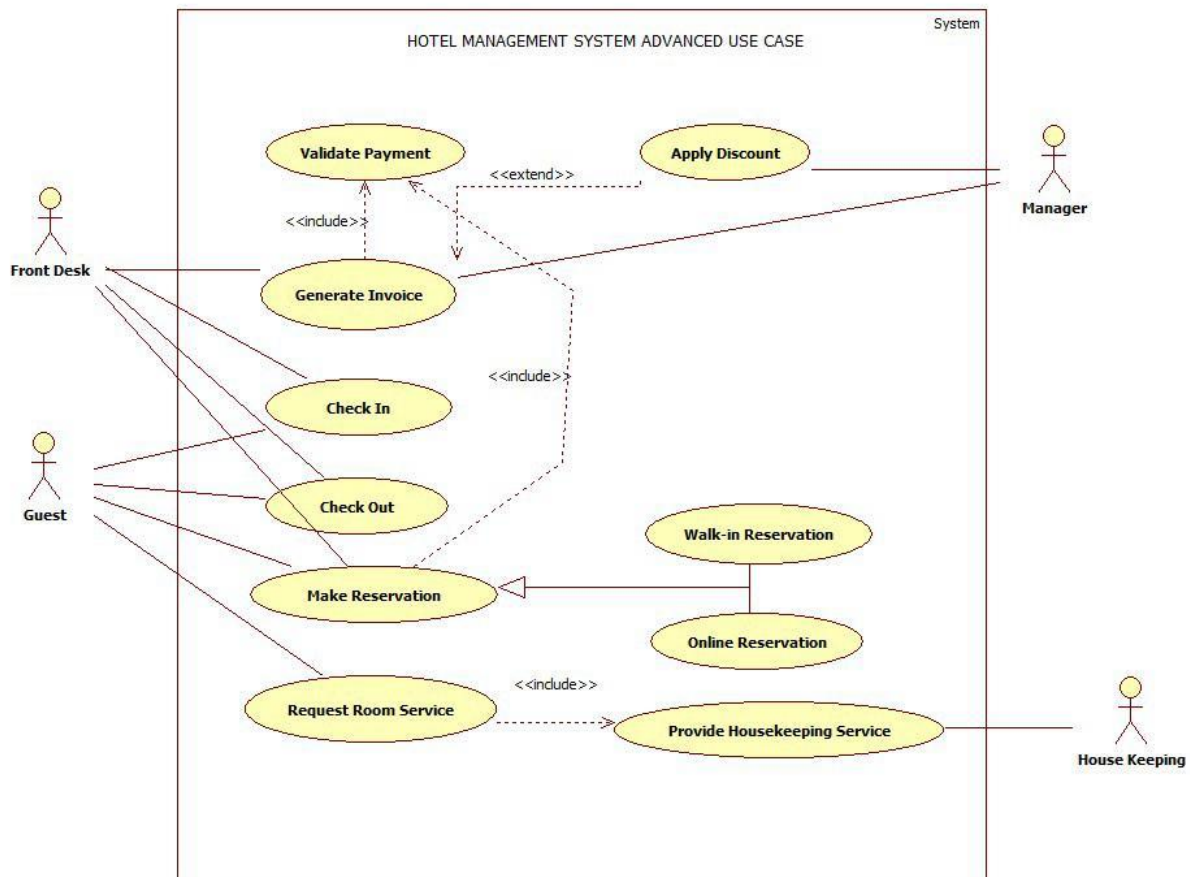


Fig 1.6

Include Relationships

These represent mandatory sub-functions.

- **Make Reservation** → *includes* **Validate Payment**
- **Check Out** → *includes* **Generate Invoice**
- **Request Room Service** → *includes* **Provide Housekeeping Service**

Extend Relationships

These represent optional or conditional tasks.

- **Validate Payment** → *extended by* **Apply Discount**
(Manager applies discounts only under special conditions)

Generalization

Shows actor specialization:

- **Reservation** is generalized into:
 - **Walk-in Reservation**
 - **Online Reservation**
- **Front Desk** is a specialized form of **Receptionist**
- **Housekeeping** is a specialized form of service provider for service requests

1.9 Sequence Diagram

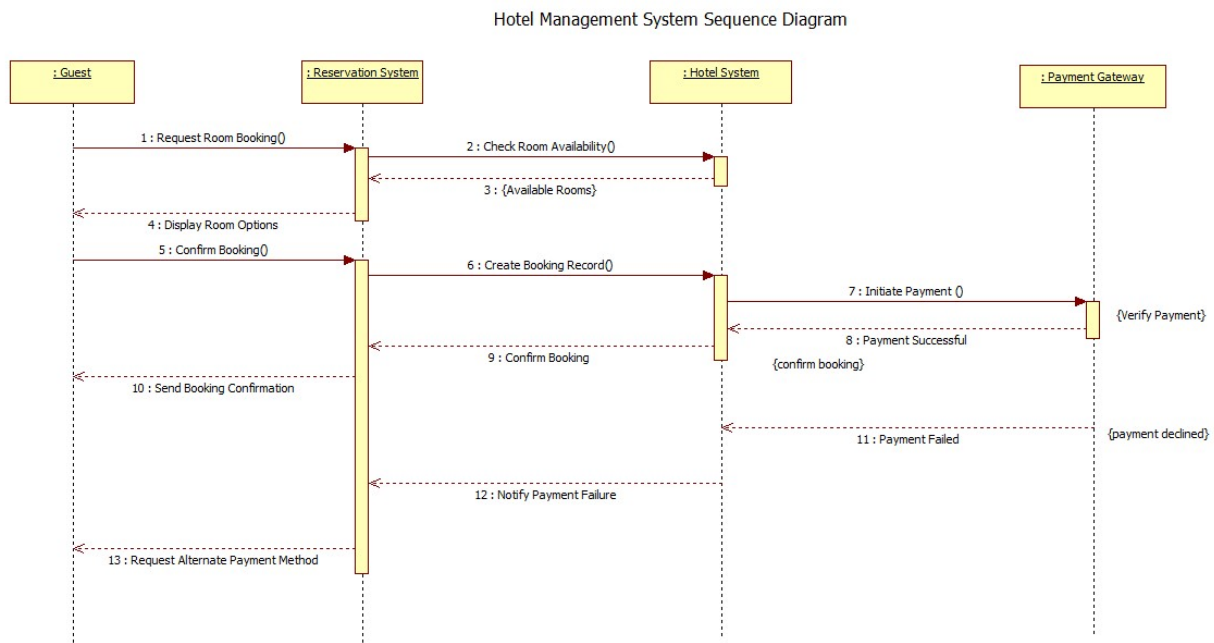


Fig 1.7

Scenario 1: Room Booking (Use Case 1)

Complete Scenario Description

1. The **Guest** initiates the booking process by sending a *Request Room Booking* message to the **Reservation System**.
2. The **Reservation System** contacts the **Hotel System** to *Check Room Availability*.
3. The **Hotel System** returns a list of *Available Rooms*.
4. The **Reservation System** displays available options to the **Guest**.
5. The **Guest** selects a room and sends *Confirm Booking*.

6. The **Reservation System** creates a *Booking Record* in the **Hotel System**.
7. The **Hotel System** initiates *Payment* by interacting with the **Payment Gateway**.
8. If the payment is successful, the **Payment Gateway** returns *Payment Successful*.
9. The **Hotel System** confirms the booking with the **Reservation System**.
10. The **Reservation System** sends *Booking Confirmation* to the **Guest**.
11. If payment fails, the **Payment Gateway** returns *Payment Failed*.
12. The **Reservation System** notifies the **Guest** of payment failure.
13. The **Guest** is prompted to use an alternate payment method.

Scenario 2: Guest Check-In and Service Request (Use Case 2)

Complete Scenario Description

1. The **Guest** sends a *requestRoom()* message to the **Reservation System**.
2. The **Reservation System** checks room availability with the **Hotel System**.
3. The **Hotel System** returns a list of available rooms.
4. The **Reservation System** sends these options back to the **Guest**.
5. Once the **Guest** confirms a room, the **Reservation System** creates a booking in the **Hotel System**.
6. The **Hotel System** responds with a *bookingId*.
7. The **Reservation System** initiates a **StaySession** object using the message `<<create>>startStay()`.
8. The **Hotel System** assigns a room and returns the room number.
9. The **Services** subsystem issues the room key (*issueKey()*) and provides it to the **Guest**.
10. During the stay, the guest can request additional services (e.g., laundry, cleaning).
11. The **Guest** sends a service request via the **StaySession** to the **Services** subsystem.

12. The Services subsystem fulfills the request and updates the stay session.
13. At checkout time, the **Guest** initiates the *checkout()* request.
14. The **Hotel System** completes the stay, settles all services, and returns confirmation.
15. The **StaySession** object is destroyed (<<destroy>>).
16. Finally, the Reservation System sends a receipt containing invoice ID and transaction ID to the Guest.

1.10 Advanced Sequence Diagram

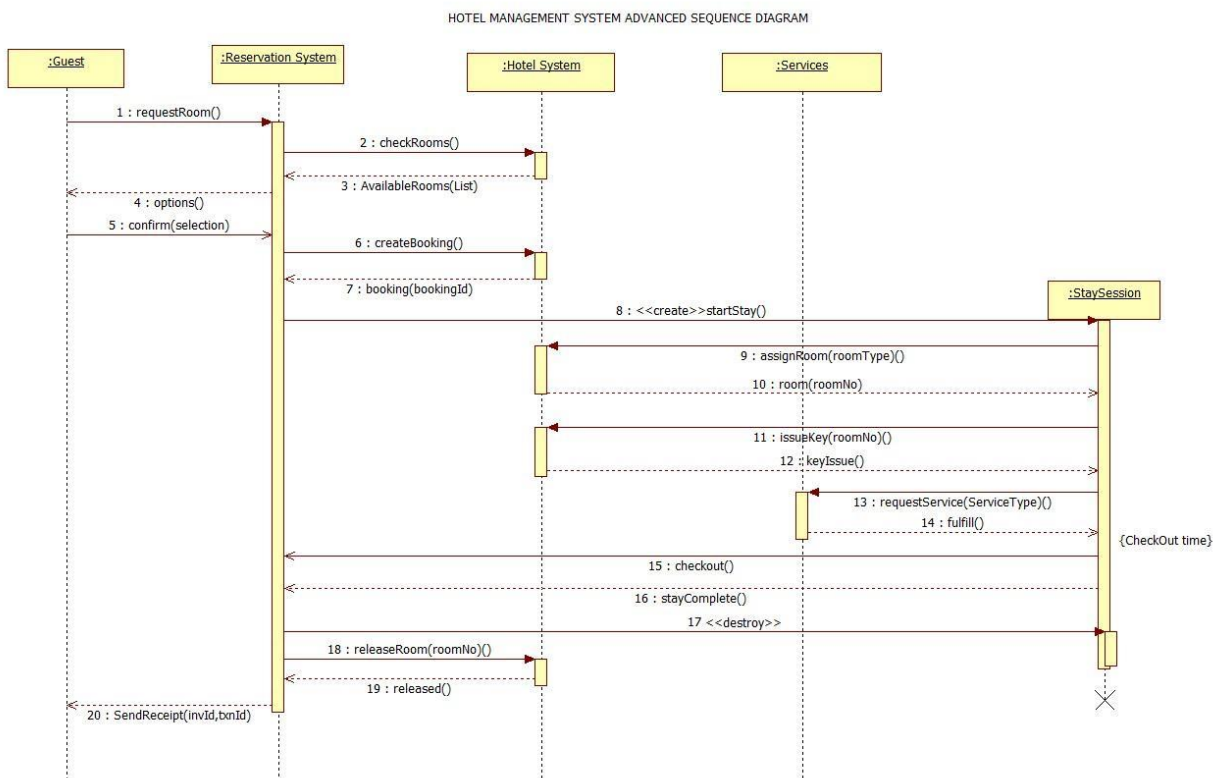


Fig 1.8

Transient Objects

These objects are **created temporarily** during the interaction and destroyed afterwards.

- **StaySession**
 - Created when the guest begins their stay.

- Maintains temporary information such as room assignment and in-stay services.
- Destroyed once checkout is complete.
- Represented using <<create>> and <<destroy>>.
- **RoomKey** (implicit from keyIssue())
 - Exists only during room allocation.
 - Not stored permanently.

Passive Objects

Objects that **do not initiate actions** but participate by storing or providing data.

- **Booking Record**
 - Stores bookingId, room number, guest details.
 - Does not control the flow, only holds data.
- **Services**
 - Responds to requests like assigning rooms or issuing keys.
 - Does not initiate any sequence on its own.

1.11 Activity Diagram

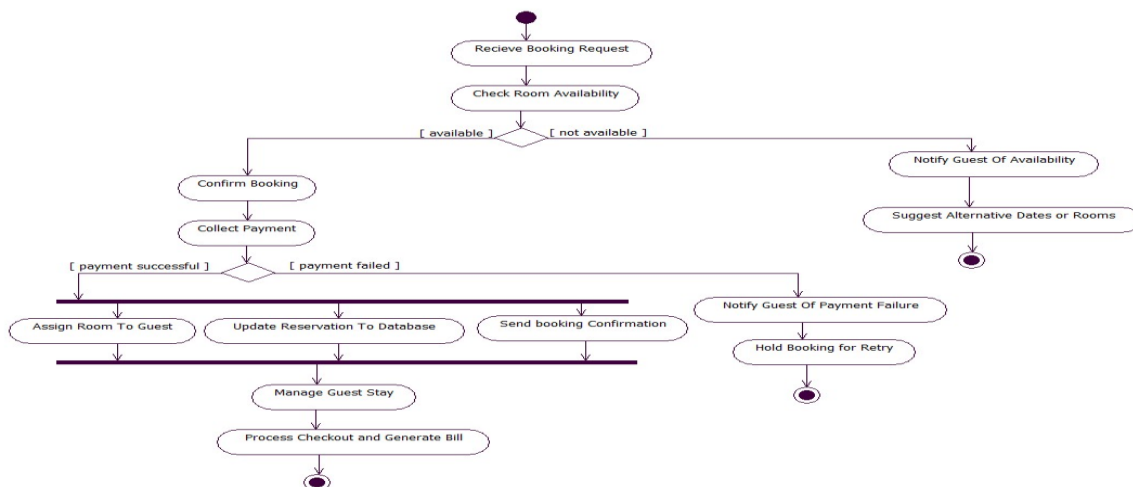


Fig 1.9

1. The process starts when the system receives a booking request from the guest.
2. The system checks room availability.
3. A decision is made:
4. If rooms are not available, the guest is notified and alternative dates/rooms are suggested.
5. If rooms are available, the system moves to confirm the booking.
6. The system then attempts to collect payment from the guest.
7. Payment status leads to two branches:
8. Payment successful:
 - a. Room is assigned to the guest.
 - b. Reservation details are updated in the database.
 - c. Booking confirmation is sent to the guest.
9. Payment failed:
 - a. The guest is notified about payment failure.
 - b. Booking is placed on hold for retry.
10. After assignment, the system proceeds to manage the guest stay.
11. Finally, the system processes checkout, generates the bill, and the activity ends.

1.12 Advanced Activity Diagram

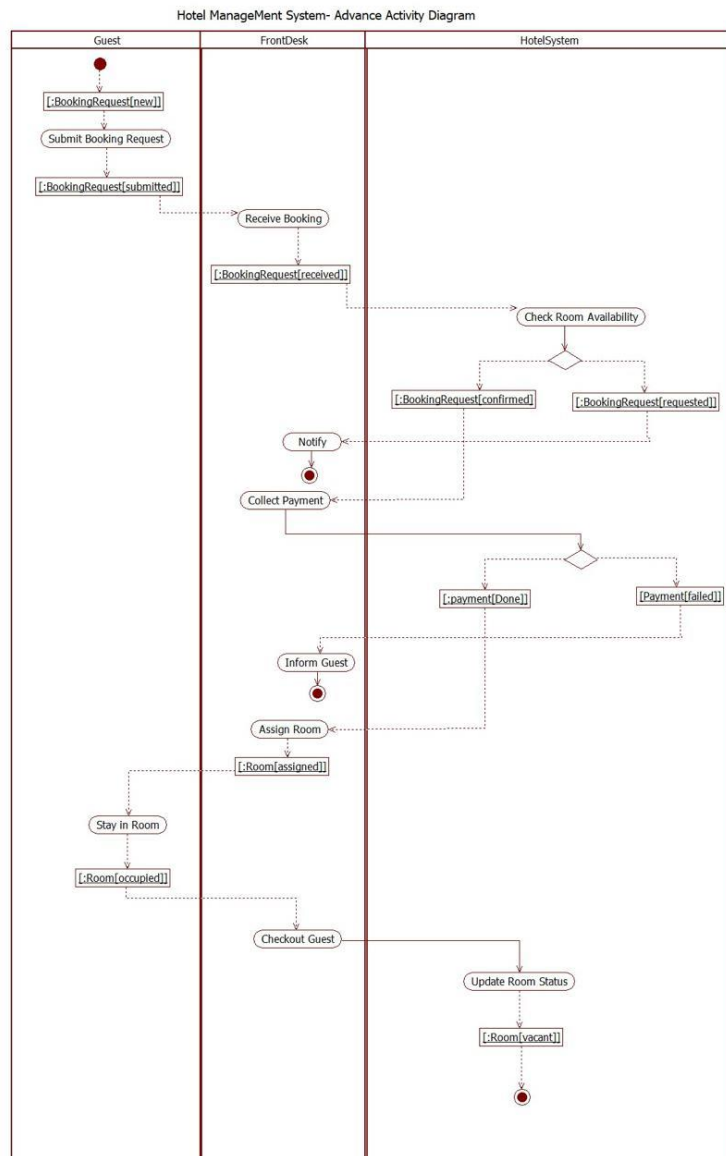


Fig 1.10

1. The **Guest** initiates the process by submitting a booking request.
2. The **Front Desk** receives the booking and acknowledges the request.
3. The **Hotel System** checks room availability and returns the result.
4. The Front Desk **notifies the guest** about room availability.
5. The Front Desk proceeds to **collect payment**.
6. Based on the payment status:
7. If **payment succeeds**, the Hotel System updates booking status.

8. If **payment fails**, the process branches to failure handling.
9. After confirmation, the Front Desk **assigns a room** to the guest.
10. The Guest then proceeds to **stay in the room**, marking the occupancy.
11. At checkout time, the Front Desk initiates **checkout processing**.
12. The Hotel System **updates room status** to vacant.
13. The activity ends after all states and updates are completed.

2.Credit Card Processing

2.1Problem Statement

The Credit Card Processing System enables secure online and offline payment transactions between customers and merchants. The system validates card details, checks available credit, authenticates customers, processes authorization requests through the payment gateway, and obtains approval from the bank. It also handles declined transactions, fraud checks, and generates transaction confirmations for both customers and merchants. The goal is to ensure fast, reliable, and secure credit card payments through proper verification and settlement mechanisms.

2.2 SRS-Software Requirements Specification

Credit Card

Problem Statement: To design and implement a Credit Card Processing system. Managing credit card manually with outdated systems creates risks such as delays, error, fraud & poor customer service. With the increasing volume of digital payments, banks & financial institutions need a secure, automated credit card processing system that can handle card applications, approvals, transactions, billing, fraud detection & customer account management efficiently.

Introduction

1.1 Purpose of the Document

The purpose of this document is to define the functional & non-functional requirements of the credit card processing system (CCPS). The system will manage the complete credit card lifecycle including issuance, transactions, billing, fraud detection, & reporting. This document will guide developers, testers, and stakeholders in ensuring the system meets business needs.

1.2 Scope of the Document

- ① Allow customers to apply for credit cards online
- ② Process transactions (purchases, cash withdrawals, online payments)
- ③ Generate monthly statements & bills
- ④ Provide fraud detection & alert mechanism
- ⑤ Enable administrators to manage credit limits, block/unblock cards, & generate reports
- Support integration with banks, payment gateways & mobile apps

The system will ensure:

- Security of sensitive financial data
- Reliability in transaction processing
- Scalability to handle millions of transactions daily

1-3 services 1. Customer Module - Apply for cards, view stmts, pay bills 2. Transaction Module - Handle real-time transaction authorization 3. Admin Module - Card Management, fraud detection & reporting 2. General Description User of the system customer (Card Holders) Bank administrators Credit card processing staff fraud detector / security officers Assumptions & Dependencies System requires integration with core banking & payment gateway Transactions must comply with PCI DSS security standards Internet & mobile connectivity required for customer access 3. Functional Requirements 1. User Authentication & Authorization Customers log in securely to their accounts Admins have role-based access 2. Credit Card Application & Approval Customers can apply online System checks eligibility & provides approval 3. Transaction Processing Authorize & process card swipes, online payments & withdrawals Update customer balance in real time 4. Billing & Stmt Generate monthly stmts Calculate interest, fees & rewards points Provide online bill payment facility	5. Fraud Detection Monitor unusual spending patterns Temporarily block suspicious accts 6. Admin Controls Approve / deny credit card applications Adjust credit limits Block / unblock cards 7. Report & Analytics Transaction reports, customer spending reports, fraud alerts Export to PDF/Excel 8. Interface Req. User Interface Web & mobile-friendly portal for customers Admin dashboard with analytics & fraud alerts Hardware Interface ATMs, POS (Point of Sale) machines & servers Software Interface Integrate with banking core system Payment gateways (Visa, Mastercard, RuPay) SMS / Email services for alerts 9. Performance Req. Should handle at least 10,000 transactions per sec Transaction approval response time should be < 2s System availability 24/7 with 99.9% uptime 10. Design Constraints Must comply with PCI DSS & RBI guidelines Encrypt (AES-256) for transaction data Limited access to sensitive customer data
---	---

7. Non functional requirements 1. Security - 2 factor Authentication (OTP, biometrics) 2. Reliability - Backup server & disaster recovery plan 3. Scalability - Cloud support for high transaction volume 4. Maintainability - Modular architecture for easy updates 5. Usability - Simple & intuitive UI for customers 8. Preliminary schedule & Budget Phase I P2 P3 P4 P5 Total Duration	1. Requirement Analysis 3 weeks 2. System Design 4 weeks 3. Development 12 weeks 4. Testing & QA 5 weeks 5. Deployment & Training 3 weeks 27 weeks (7 months) Budget €4,00,000 €15,00,000 €3,00,000 €2,00,000 €6,00,000 €30,00,000 €43,00,000
--	--

2.3 Class Diagram

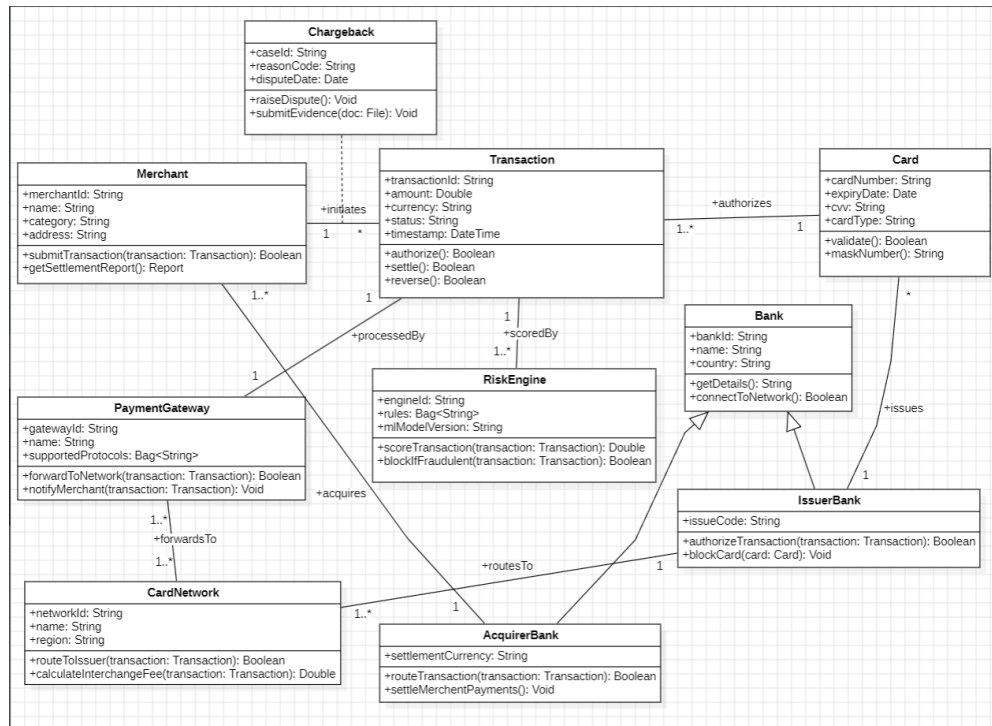


Fig 2.1

The **Merchant** class initiates transactions and sends information to the system. The **Transaction** class stores transaction details such as amount, currency, status, and timestamp. Each transaction is validated and processed with the help of the **Card** class, which contains card details like card number, expiry date, CVV, and card type.

The **PaymentGateway** class forwards transactions between the merchant and the banking network, and communicates with the **CardNetwork** for routing. The **IssuerBank** authorizes or declines the transaction based on card validity and available credit, while the **AcquirerBank** handles settlement for the merchant.

The **RiskEngine** evaluates transactions for fraud detection using rules and ML models. When disputes arise, the **Chargeback** class manages dispute raising and evidence submission. The **Bank** class maintains bank details and connects to the card network. Together, these classes represent a complete ecosystem for validating transactions, detecting fraud, authorizing payments, and settling funds securely.

2.4 Advanced Class Diagram

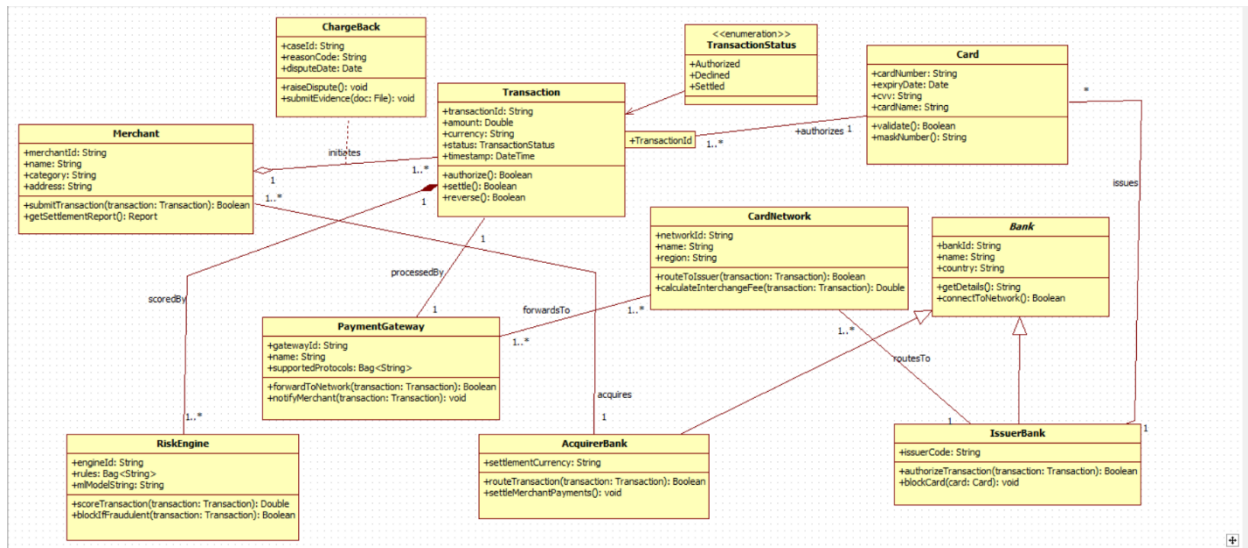


Fig 2.2

The advanced class diagram represents a complete credit card processing ecosystem involving merchants, banks, networks, and fraud detection components. The **Transaction** class is central, storing transaction details and supporting operations such as authorization, settlement, and reversal. A **Merchant** initiates multiple transactions, while the **Card** class holds sensitive card information and is linked to the **IssuerBank**, which authorizes or blocks the card.

The **PaymentGateway** forwards transactions to the **CardNetwork**, which routes them to the correct issuer and calculates interchange fees. The **AcquirerBank** settles payments for the merchant, whereas the **RiskEngine** scores transactions for fraud and can block risky ones. The **Chargeback** class manages disputes raised for incorrect or fraudulent transactions. Together, these components model a realistic financial workflow, ensuring secure routing, authorization, fraud analysis, and settlement within the credit card processing system.

2.5 State Diagram

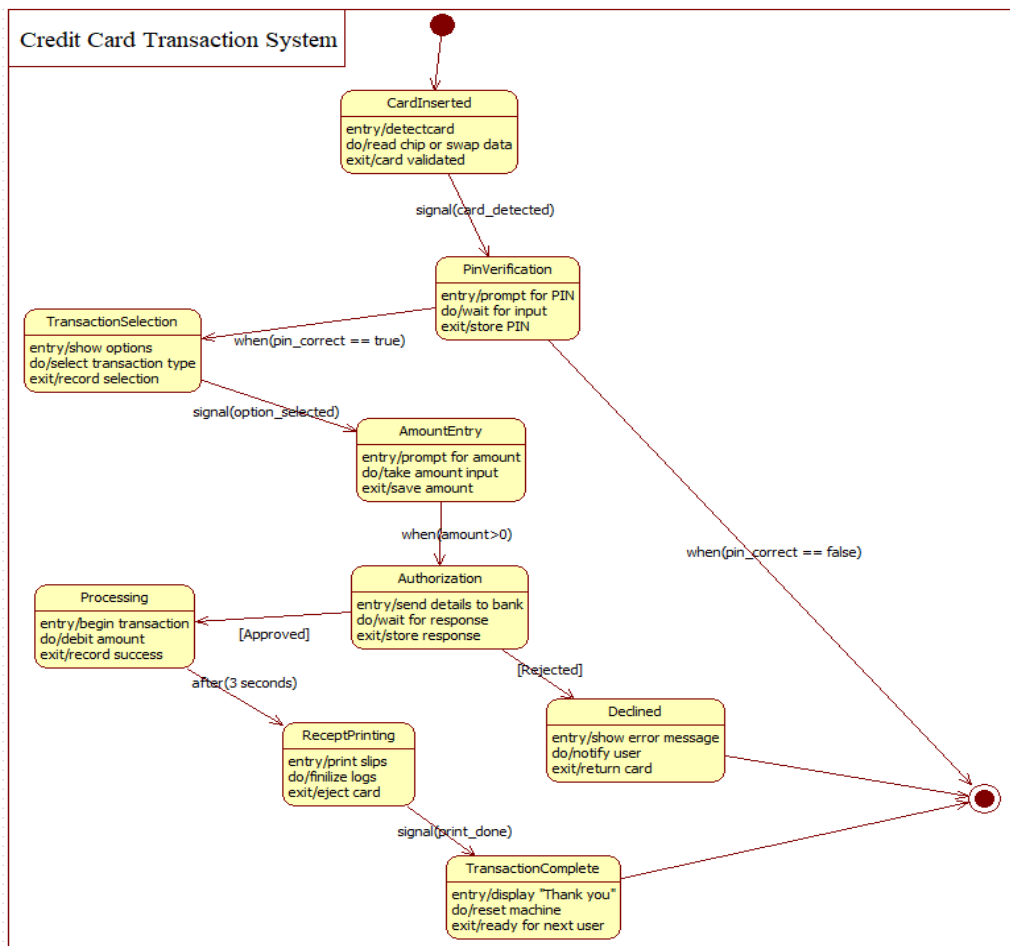


Fig 2.3

This state diagram describes the main states of a credit card transaction. The system begins by receiving a transaction request and verifying card details. If invalid, the transaction is immediately rejected. If valid, the system checks the available credit limit.

With sufficient limit, the transaction enters the *Authorized* state and the amount is debited. After this, the system updates all records and sends notifications before generating the monthly statement. With insufficient credit, the transaction goes to *Declined* state and the user is notified.

2.6 Advanced State Diagram

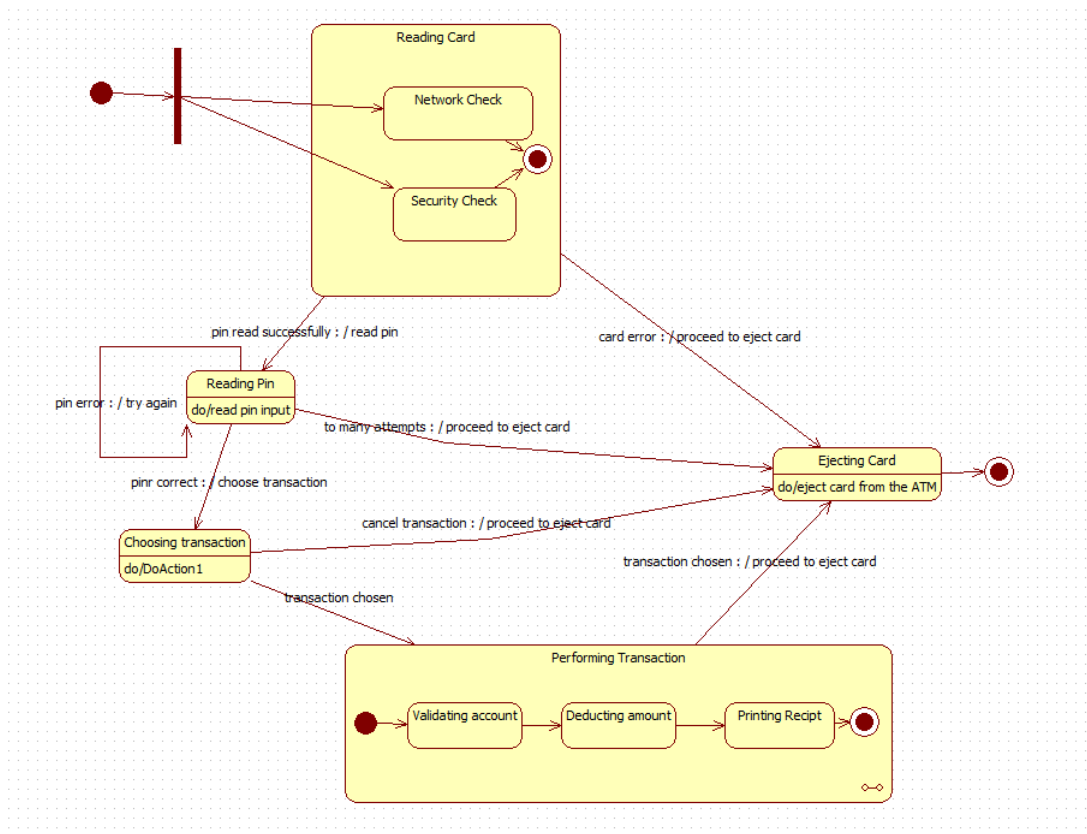


Fig 2.4

The advanced state diagram covers ATM-style credit card operations. The flow begins with card insertion followed by network and security checks. The PIN verification state allows retries and failure transitions. Once the PIN is correct, users select a transaction type and enter the amount.

The authorization state communicates with the bank for approval. If approved, processing and receipt printing occur. If rejected, the user is notified and the card is returned. The flow ends with transaction completion and machine reset.

2.7 Use Case Diagram

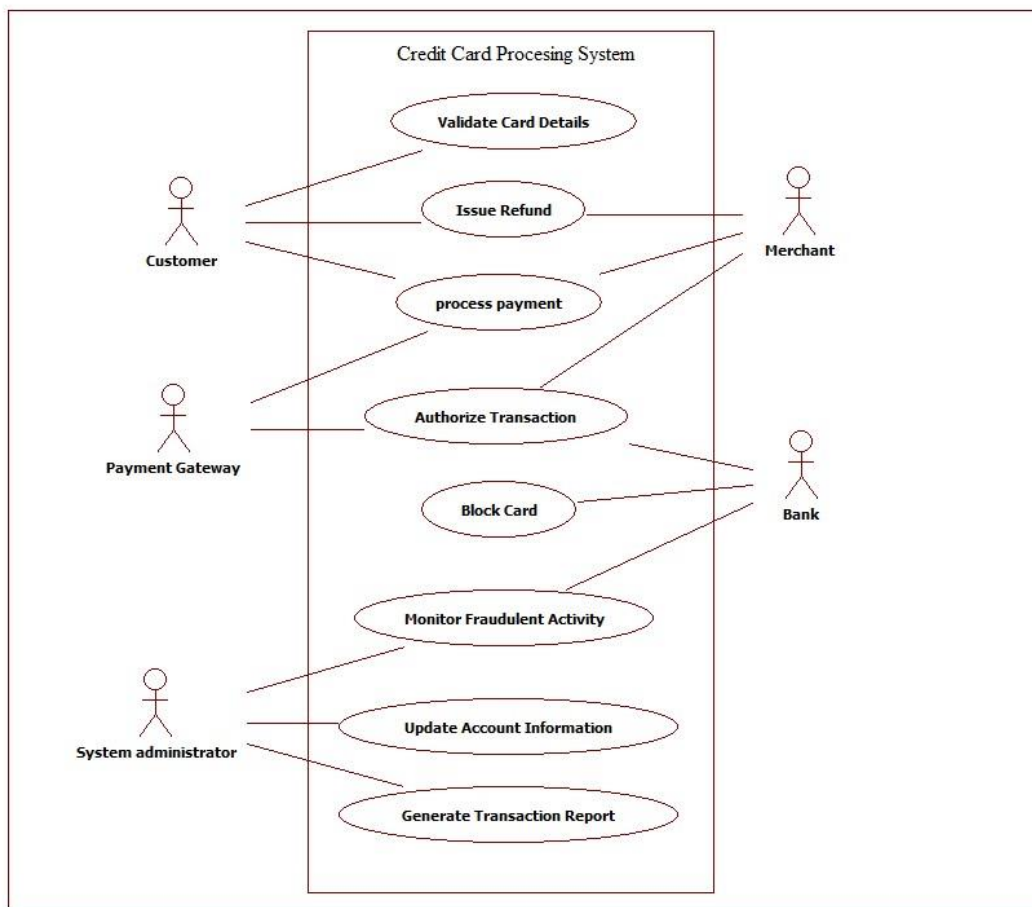


Fig 2.5

The simple use case diagram shows the high-level interactions between system actors. The **Customer** initiates payments, requests refunds, and validates card details. The **Merchant** processes payments and interacts with the bank for authorization. The **Payment Gateway** verifies card information and forwards authorization requests. The **Bank** authorizes transactions and blocks cards in case of suspicious activity. The **System Administrator** monitors fraudulent activities, updates account details, and generates reports.

The system supports essential use cases like card validation, payment processing, transaction authorization, card blocking, fraudulent activity monitoring, and report generation.

2.7 Advanced Use Case Diagram

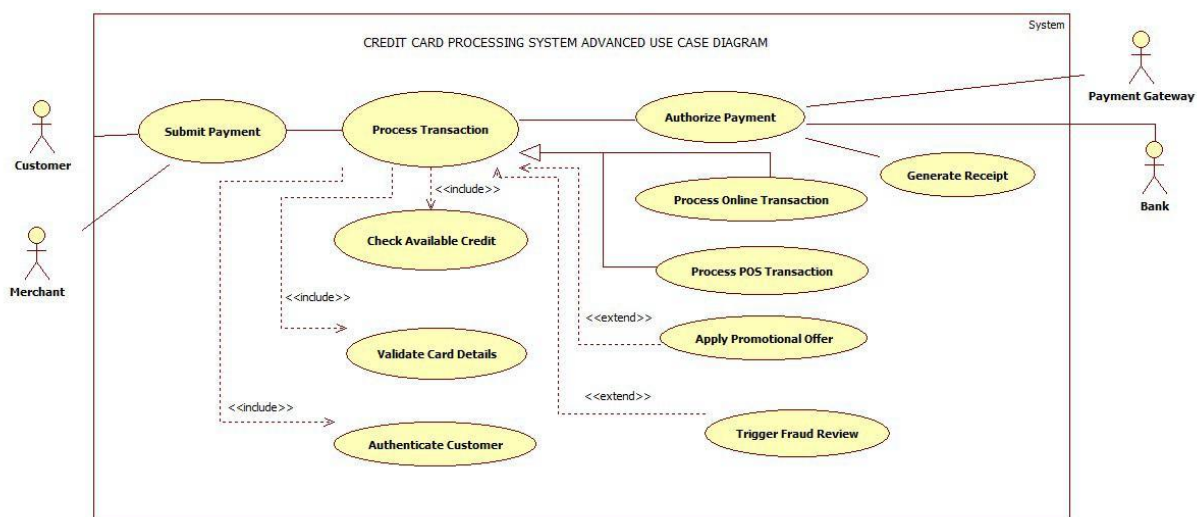


Fig 2.6

The advanced use case diagram provides a detailed set of operations. The **Customer** submits a payment request, while the **Merchant** initiates transaction processing. The **Process Transaction** use case includes verifying available credit, validating card details, and authenticating the customer. The **Authorize Payment** use case further includes online transaction processing and POS transaction processing.

Promotional offers may extend from transaction processing, and fraud review may extend from authorization. The **Payment Gateway** generates receipts, while the **Bank** assists in authorization. This diagram captures a comprehensive flow of validation, authorization, fraud detection, and settlement, reflecting real-world processing steps.

2.8 Sequence Diagram

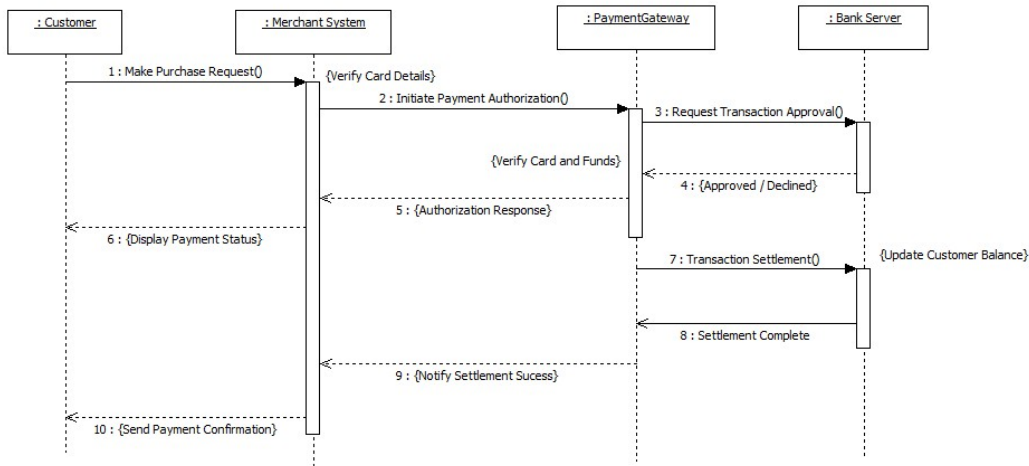


Fig 2.7

The simple sequence diagram shows the communication between the four main entities. The **Customer** makes a purchase request, and the **Merchant System** verifies card details and sends an authorization request to the **Payment Gateway**. The gateway forwards the request to the **Bank Server**, which checks card validity and funds availability.

The bank sends back an authorization response (approved or declined). If approved, the gateway proceeds with the settlement process, and the merchant updates the customer with the payment status. This ensures proper authorization, fund verification, and completion of the transaction.

2.9 Advanced Sequence Diagram

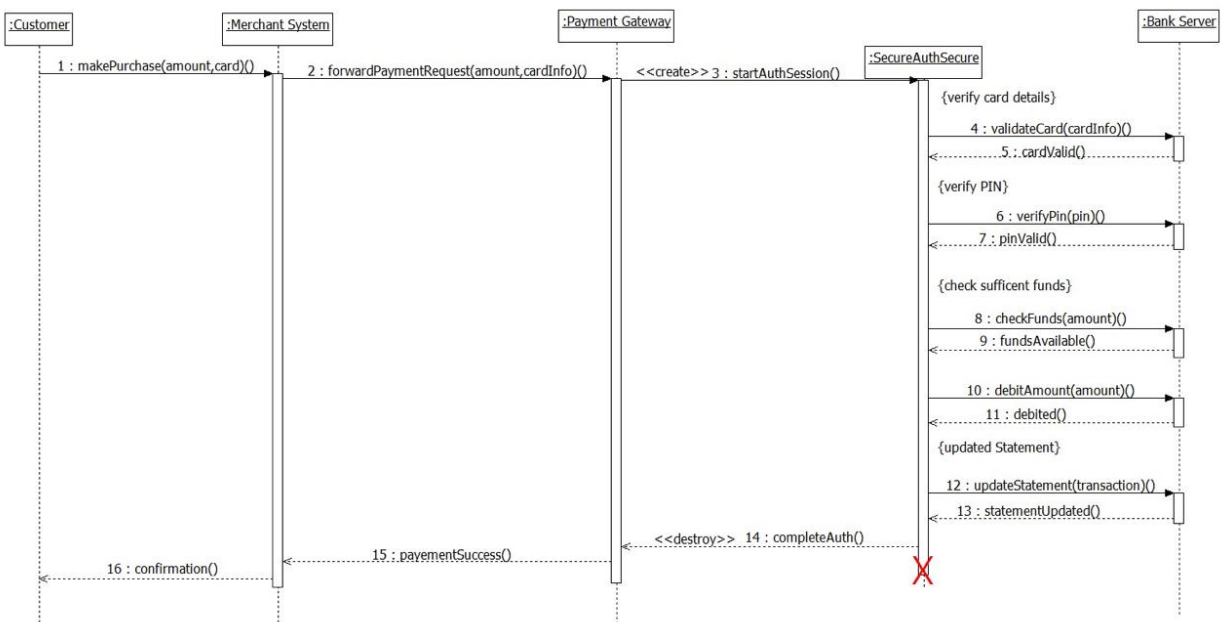


Fig 2.8

The advanced diagram includes more detailed authentication and verification steps. The **Customer** initiates a payment request, and the **Merchant System** forwards the request to the **Payment Gateway**, which creates an authentication session. The **SecureAuthSecure** component validates the card, verifies the PIN, checks funds, debits the amount, and updates the account statement through the **Bank Server**.

After all steps succeed, the authentication session is destroyed, the payment gateway confirms success to the merchant, and the customer receives final confirmation. This detailed diagram reflects deeper security and fraud-prevention mechanisms.

2.10 Activity Diagram

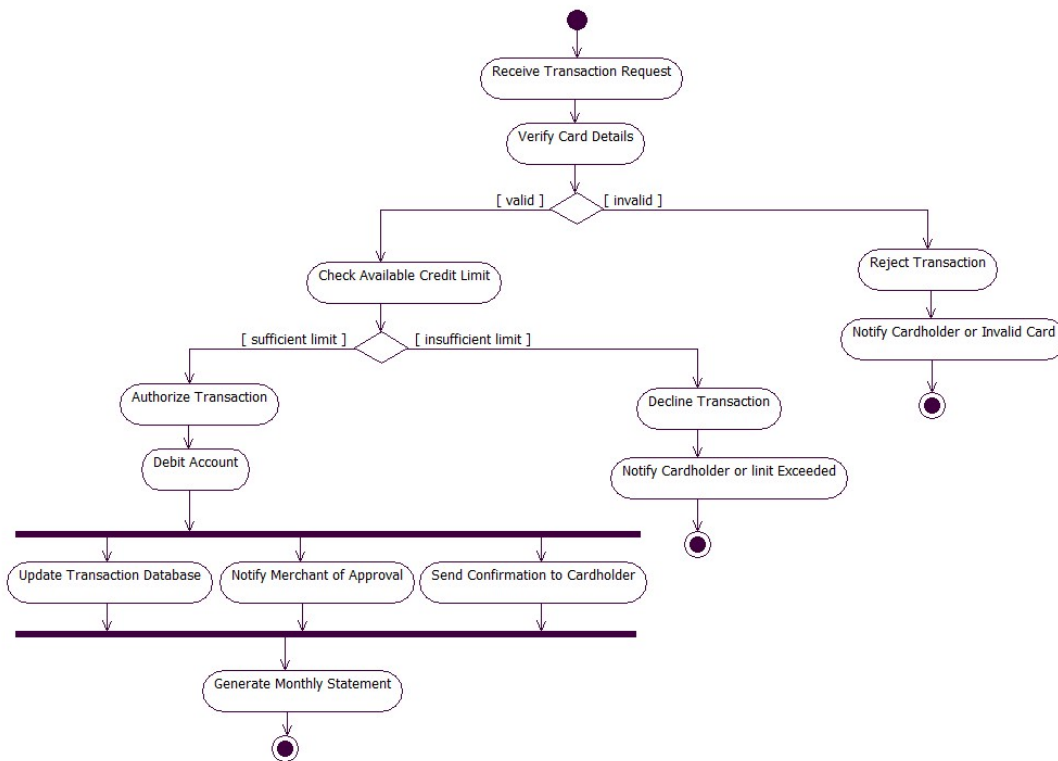


Fig 2.9

The simple activity diagram outlines basic transaction flow.

The process begins by receiving a transaction request and verifying card details. If the card is

invalid, the transaction is rejected. If valid, the system checks the available credit limit. If the limit is sufficient, the transaction is authorized and the account is debited.

Next, the system updates the transaction database, notifies the merchant, and sends confirmation to the cardholder. Finally, a monthly statement is generated. This diagram represents simple decision-based payment processing.

2.11 Advanced Activity Diagram

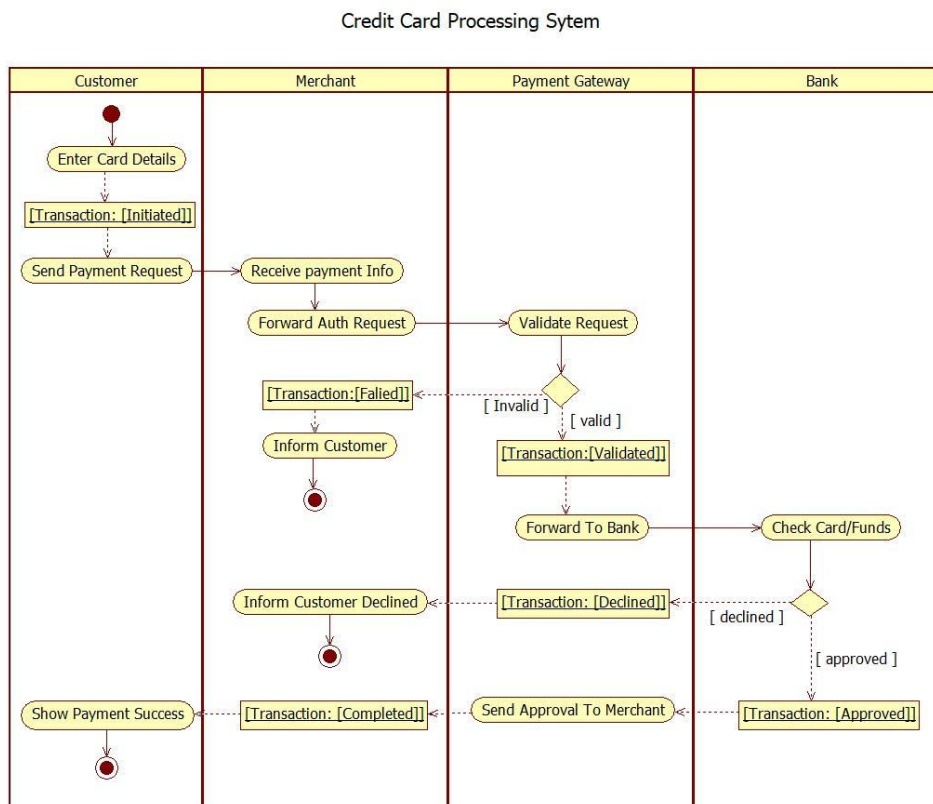


Fig 2.10

The advanced activity diagram includes more detailed swimlane-based flow. The transaction starts with card reading and network/security checks. If errors occur, the card is ejected. On successful reading, the PIN is verified with retry options. A valid PIN leads to transaction selection.

The **Performing Transaction** activity includes validating the account, deducting the amount, and printing the receipt. Any errors trigger card ejection. This diagram highlights detailed ATM-style credit card transaction behavior with error handling and multi-step verification.

3. Library Management System

3.1 Problem Statement

The Library Management System (LMS) is designed to automate core library operations such as searching books, borrowing, returning, reserving, renewing, and paying fines. The system maintains information about books, members, staff, and loan records. It ensures efficient catalog management, availability checking, overdue fine calculation, and updating of inventory. LMS also supports staff functions like issuing and accepting returned books, managing memberships, and updating catalog records. The goal is to provide a smooth, accurate, and organized library workflow for both users and staff.

3.2 SRS-Software Requirements Specification

1. Introduction

1.1 Purpose of the Document

The purpose of this document is to define the functional & non-functional requirements of LMS. The system will manage library resources by automating processes like book cataloging, borrowing, returning, reservations, & fine calculation. It will serve as a reference for developers, testers & stakeholders to ensure the project's success.

1.2 Scope of the Document

The LMS will

Manage books, journals & digital resources.

Maintain member details & borrowing history.

Allow book issue, return, & reservation.

Calculate overdue fines automatically.

Provide search functionality for books & members.

Generate reports for librarians.

The system will ensure

Fast processing of library transactions.

Reduced manual errors.

Easy access to library resources for members.

1.3 Overview

The system will have 3 main modules.

1. Admin / Librarian Module - Manage books, members & transactions.

2. Member Module - Search, book request, borrow, & return.

3. Report Module - Generate statistics about usage, fines & inventory.

General Description

Users of the system

Librarians (Admins)
Library Members (Students, Teachers, Researchers)

Assumptions & Dependencies

Database required to store books, members & transaction records

Internet / LAN connectivity for multi-user access

Barcode / QR scanners can be used for faster processing

Functional Req

1. User Authentication

Librarians & members login, securely

2. Book Management

Add, update, delete book records
Categorize books by subject, author & availability

3. Member Management

Register new members

Maintain borrowing history

4. Book Issue & Return

Issue books to members & record the due date
Update system on return & calculate overdue fines

5. Search & Reservation

Members can search for books by title, author, ISBN or category

Reservation of unavailable books

6. Fine Calculation

Automatic fine calculation based on overdue days

7. Reporting

Reports for issued / returned books

Fines collected & overdue books

Member activity reports

4. Interface Requirements

• User Interface

User friendly web & desktop, PC
Search bar, navigation menus, role-based dashboard

• Hardware Interface

Works on standard PC's with min Intel i3, 4GB RAM

Barcode / QR code scanner support

• Software Interface

Database: MySQL / PostgreSQL

OS: Windows / Linux

Language: Java / Python / PHP, with web framework

5. Performance req

System should handle 500 concurrent users

Book search result should load in < 2s

Database should support upto 1 million records

6. Design Constraints

Must comply with data protection regulations

Only librarians can modify book & member data

Maintain backup of records regularly

7. Non functional attributes

Security: Role based access control, secure login

Reliability: Daily database backup, 99.5% uptime

Scalability: support for growing no of members & resources

Usability: simple & intuitive design for non-technical users

Maintainability: Modular structure for easy updates

Preliminary Schedule & Budget

Phase	Task	Duration	Cost
P1	Req. Analysis	2 weeks	£1,50,000
P2	System Design	3 weeks	£6,00,000
P3	Development	5 weeks	£11,00,000
P4	Testing & QA	2 weeks	£45,000
P5	Deployment & Training	2 weeks	£1,45,000
	Total	12 weeks	£19,40,000

3.3 Class Diagram

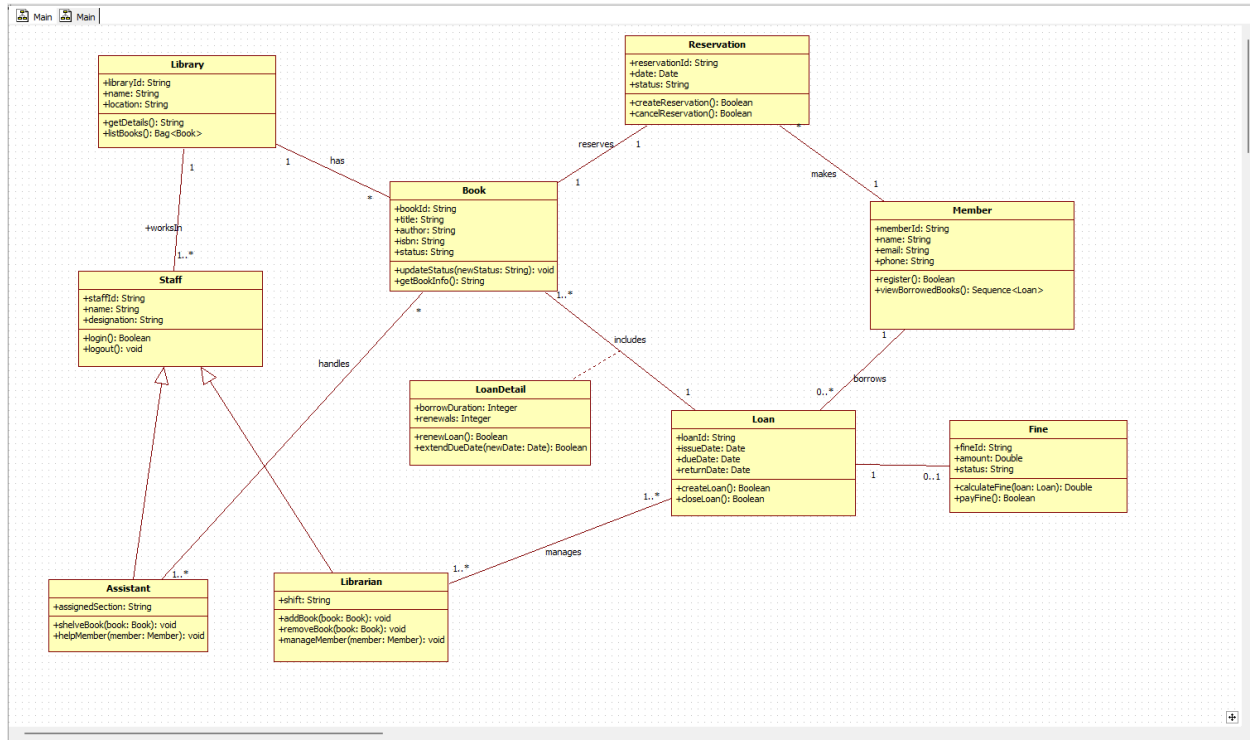


Fig 3.1

The class diagram models all major entities of the Library Management System. The **Library** contains multiple books and staff members. The **Book** class stores details such as title, author, ISBN, and status (Available, Borrowed, Reserved). **Members** borrow books through **Loan** objects, which record issue date, due date, and return date. **LoanDetail** captures borrowing duration and renewals.

Staff are modeled through the **Staff** class, which has two specializations: **Librarian** and **Assistant**, each handling tasks such as adding/removing books or helping members. The **Reservation** class stores reservation requests. The **Fine** class manages fine amount, status, and calculations when books are returned late.

Together, these classes represent the complete structural framework required for managing library transactions, membership, staff actions, and inventory.

3.4 Advanced Class Diagram

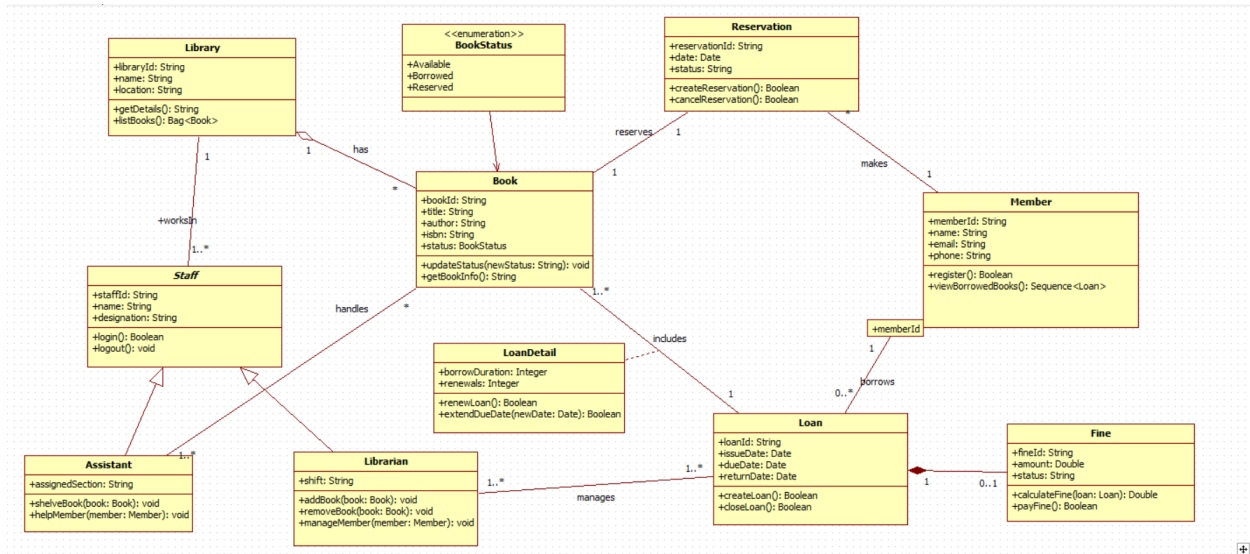


Fig 3.2

The advanced class diagram represents all major components and relationships within the Library Management System. The **Library** class maintains information about the library and has multiple **Book** objects. Each **Book** includes details such as title, author, ISBN, and a status from the **BookStatus** enumeration (Available, Borrowed, Reserved).

Members are represented through the **Member** class, which stores user information and allows viewing of borrowed books. A member can borrow multiple books through the **Loan** class, which records issue date, due date, return date, and is linked with **LoanDetail** for handling renewals and borrow duration. Overdue returns generate a **Fine**, which stores fine amount, status, and provides operations to calculate and pay fines.

Reservations are captured using the **Reservation** class, allowing members to reserve a book when unavailable. The **Staff** class models employees working in the library and is specialized into **Librarian** and **Assistant**. Librarians manage adding/removing books and member operations, while Assistants help with shelving and member support.

Overall, this advanced diagram shows a complete structural model capturing inventory management, borrowing, returning, reservations, renewals, fines, and staff operations necessary for an efficient and automated library system

3.5 State Diagram

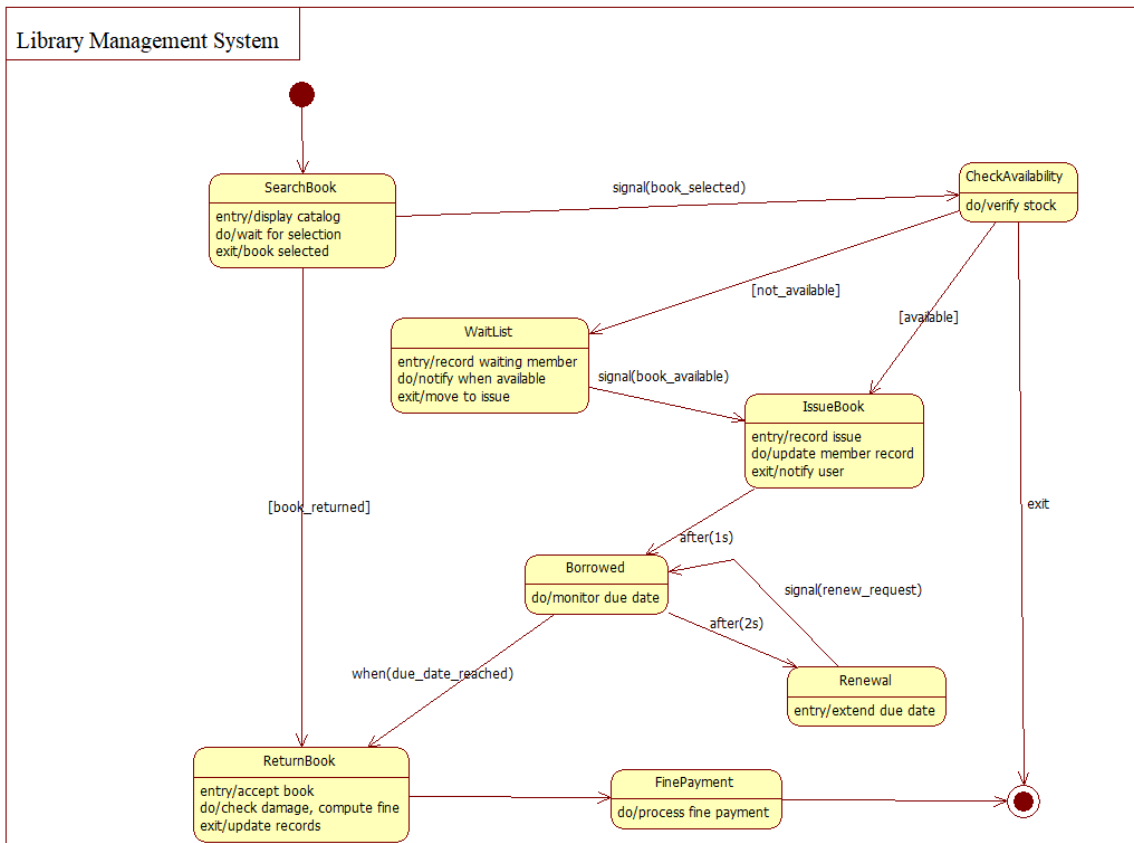


Fig 3.3

The simple state diagram shows the main states of a book request.

The process begins with a book request, followed by member ID verification. If the ID is valid, the system checks book availability. If available, the book is issued and the borrow record updated. If unavailable, the user is notified or added to a waitlist. After reading, the return state updates book inventory and handles fine calculation if overdue. This diagram expresses the lifecycle of a single borrow cycle.

3.6 Advanced State Diagram

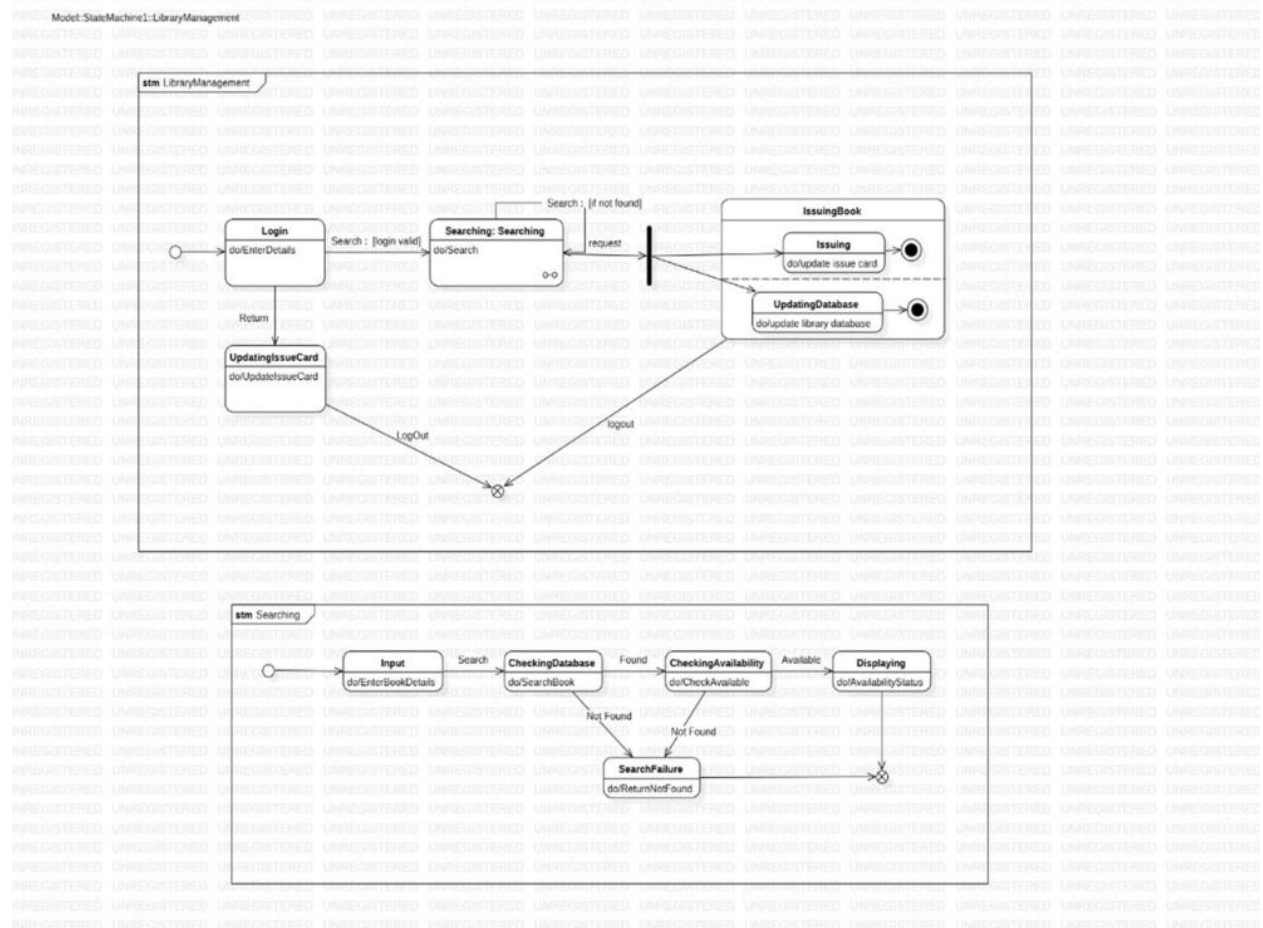


Fig 3.4

The advanced state diagram models detailed transitions for searching, issuing, borrowing, renewing, and returning books.

The process begins with **SearchBook**, followed by availability checking. If not available, the system moves to the **WaitList** state. If available, the system proceeds to **IssueBook**, then transitions to the **Borrowed** state where the due date is monitored. Members may request renewal, leading to the **Renewal** state. After reading, the user returns the book, triggering the **ReturnBook** state, which handles fine assessment and updates records. This diagram provides a more realistic model of the library's operational states.

3.7 Use Case Diagram

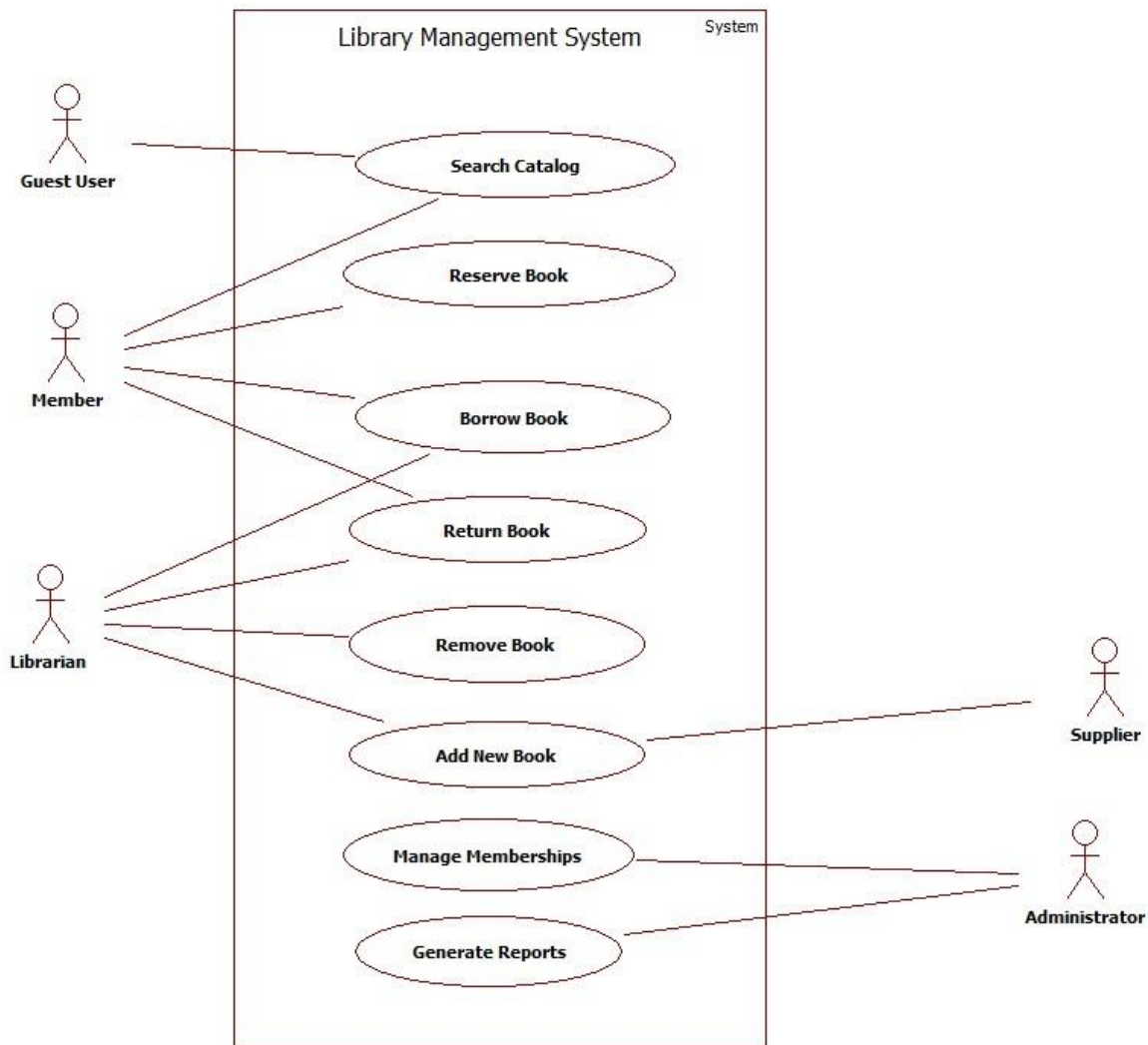


Fig 3.5

The use case diagram identifies different actors and their interactions with the system. **Members** can search, reserve, borrow, renew, return books, and pay fines. **Librarians** manage circulation tasks, issue books, accept returns, remove or add books, and manage member records. **Suppliers** provide new books, and **Administrators** handle catalog management, inventory updates, and send due-date reminders. **Guest Users** can search the catalog without borrowing privileges. The diagram clearly shows the major functionalities required for efficient library operations.

3.8 Advanced Use Case Diagram

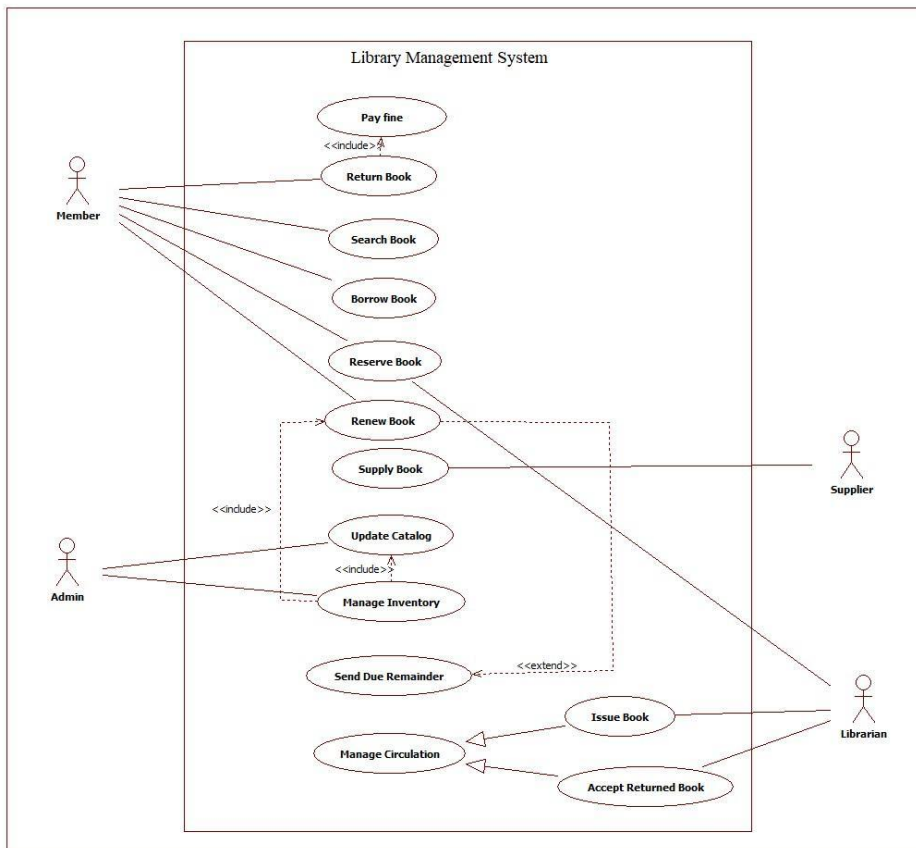


Fig 3.6

The advanced use case diagram captures all major interactions between different actors and the Library Management System. **Members** can search, reserve, borrow, renew, and return books, as well as pay fines. **Librarians** handle issuing and accepting returned books, managing memberships, adding or removing books, and maintaining the catalog. **Administrators** manage inventory, update catalog records, and generate system reports. **Suppliers** provide new books to update the catalog.

Some use cases like *Pay Fine* and *Update Catalog* are included within larger operations such as *Return Book* and *Manage Inventory*. The *Send Due Reminder* use case may extend the borrowing process to notify members before due dates. This advanced diagram clearly shows a complete set of functional requirements, covering user services, staff responsibilities, and system-level management within the library.

3.9 Sequence Diagram

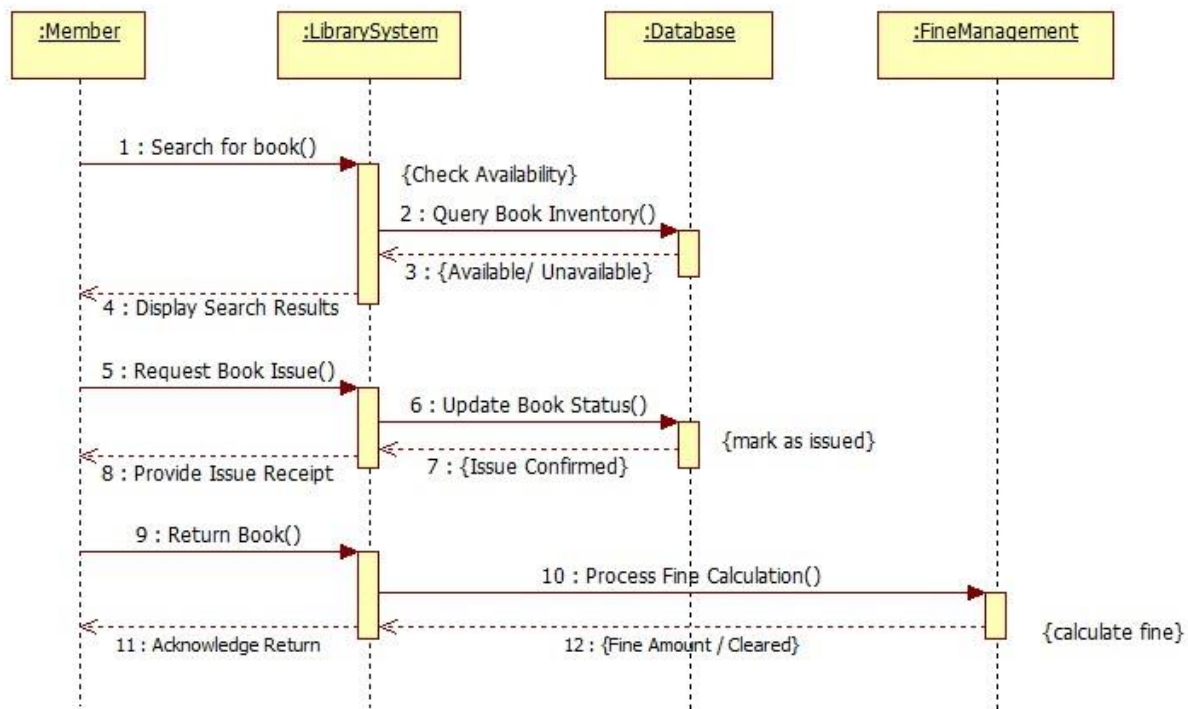


Fig 3.7

The basic sequence diagram describes the steps involved when a member searches and issues a book.

The **Member** searches for a book, the **Library System** queries the **Database**, and availability status is returned. If available, the member requests issue. The system updates book status, confirms issue, and later processes fine calculation during book return through the **Fine Management** component. Finally, the system acknowledges the return and updates records. This sequence captures a complete borrow–return cycle.

3.10 Advanced Sequence Diagram

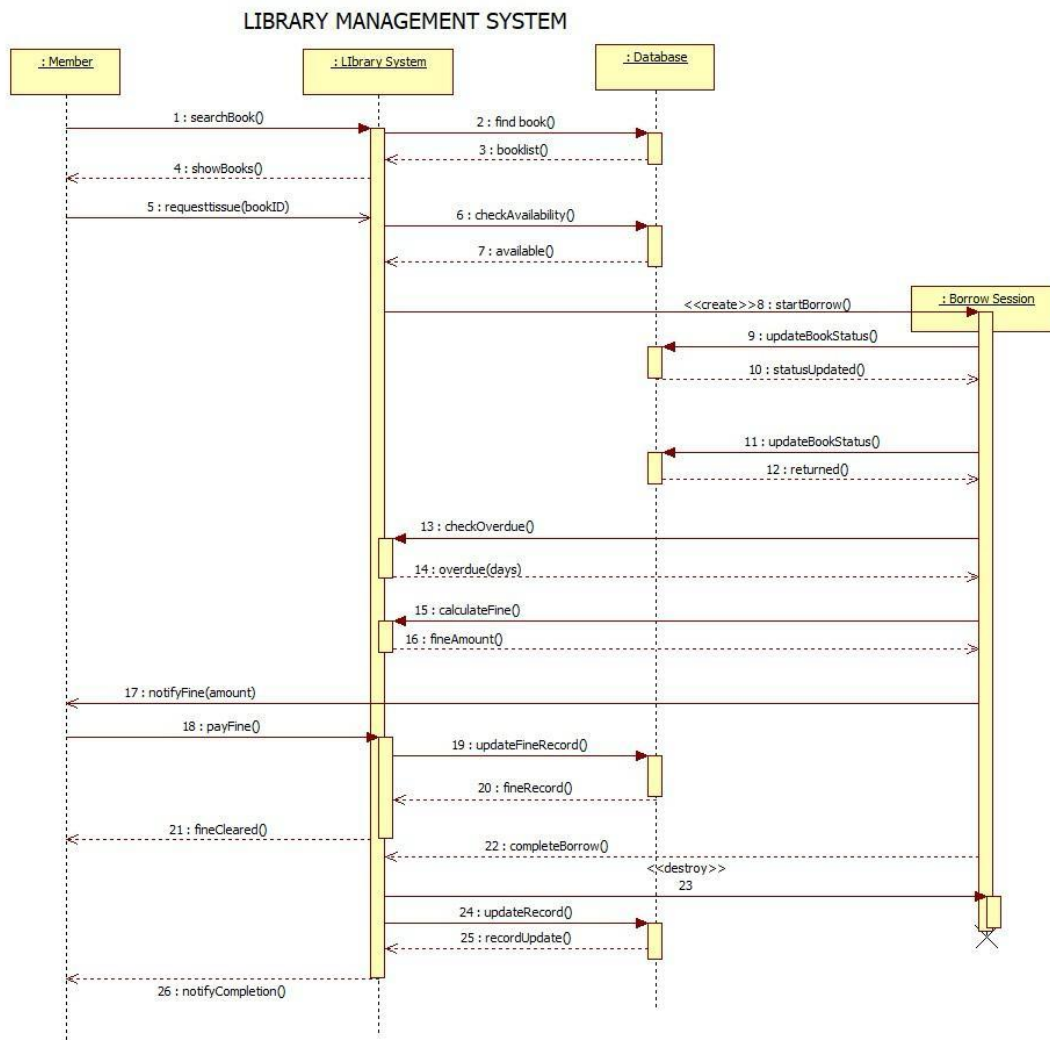


Fig 3.8

The advanced sequence diagram provides detailed interactions for issuing and returning books. The **Library System** interacts with the **Database** to find the book, check availability, and update status. A new **Borrow Session** is created to track loan duration. When the book is returned, overdue days are calculated, fines are computed, and fine records are updated. After fine clearance, the borrow session ends, and the library database updates the final return status. This detailed flow shows complete handling of search, issue, renew, return, and fine processing.

3.11 Activity Diagram



Fig 3.9

The activity diagram shows user actions and system responses during the book borrowing process.

The flow starts with a member choosing a book and going to the front desk. The librarian verifies the member ID. If invalid, the member is informed, and the process stops. If valid, book availability is checked. If unavailable, the member is notified. If available, the book is issued, and a borrow record is created. After reading, the member returns the book. The system checks for overdue fines, processes fine payments if needed, and finally reshelves the book. This diagram covers the complete operational workflow.

3.12 Advanced Activity Diagram

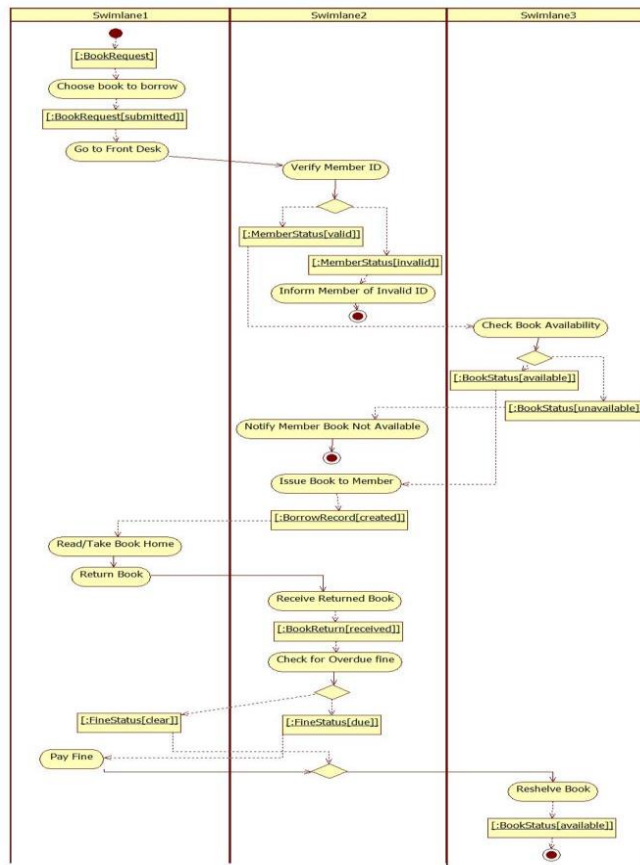


Fig 3.10

The advanced activity diagram represents the complete workflow of borrowing, returning, and renewing books in the library. The process begins when a member searches for a book, after which the system checks its availability. If the book is not available, the member is added to a waitlist. If available, the book is issued and the borrow record is created.

During the borrowing period, the system monitors the due date. Members may request renewal, after which the due date is extended. When the book is returned, the system checks for overdue days, calculates any fine, processes fine payment if applicable, and updates member and inventory records. The workflow ends by reshelving the book and marking it as available again. This activity diagram captures all detailed operations involved in issuing, borrowing, renewing, and returning books efficiently.

4. Stock Maintenance System

4.1 Problem Statement

The Stock Maintenance System is designed to manage all operations related to inventory, including adding new stock, updating quantities, handling customer orders, generating reports, and coordinating with suppliers. The system ensures accurate tracking of stock levels, validates item availability during requests, updates records when stock is added or deducted, and generates reports for decision-making. It also supports order placement, stock movement management, supplier coordination, and warehouse operations.

4.2 SRS-Software Requirements Specification

Preliminary Schedule & Budget			
Phase 1	Req. Analysis	2 weeks	£1,50,000
P2	System Design	3 weeks	£6,00,000
P3	Development & Tests	8 weeks	£11,00,000
P4	Testing & UAT	2 weeks	£45,000
P5	Deployment & Training	2 weeks	£1,45,000
	Total	15 weeks	£19,40,000

Stock maintenance system
To design & implement a Stock Maintenance System that automates the tracking, updating & management of stock levels for businesses. Manual stock maintenance often leads to errors, delays, theft & mismanagement, causing losses & inefficiency. An automated system is needed to record stock entries, monitor inventory in real-time, generate alerts for low stock, maintain supplier & transaction details & provide analytical reports, ensuring smooth & reliable stock control.

SRS

1. Introduction

Purpose
The purpose of this document is to specify the requirements for the development of a Stock Maintenance System (SMS). This system will help businesses track inventory, manage stock levels, generate reports & minimize human errors in stock handling.

1.2 Scope

The Stock Maintenance System will:
- Maintain details of items, suppliers & transactions.
- Record purchases & sales automatically.
- Provide real-time stock updates & low stock alerts.

Generate inventory & sales reports for analysts.
Ensure secure login for Admin & staff users.

1.3 Overview

The system consists of:

- Admin Module: Manage stock, suppliers, reports & users.
- Staff Module: Record stock-in, stock-out & sales.
- Database: Stores all stock, supplier & transaction details.
- Reports Module: Provides daily, monthly & annual summaries.

2 General Description

Users: Admin, Staff
System Environment: Web/Browser, Application with database support.

Constraints

- Only authorized users can access.
- Should be scalable for diff. business sizes.

Assumptions

- Users have basic computer knowledge.
- Stock transactions are genuine & verified.

3 Functional Requirements

3.1 User Management

- Admin can create, update (or) delete user accounts.
- Logins authentication for Admin & Staff.

3.2 Stock Management

- Add new items with details (ID, name, category, price, quantity, supplier).
- Update stock levels when purchase/sales occur.
- Delete discontinued items.

3.3 Supplier Management

- Maintain supplier information (name, contact, add).
- Link suppliers to purchased stock.

Transaction Management

- Record stock purchases
- Record sales / stock-out details
- Generate Invoices

Reports & Alerts

- Generate daily/weekly/monthly stock reports
- Low stock alerts when quantity falls below threshold
- Sales vs purchase analysis

(4) Interface Req.

User Interface

- Simple GUI with menu options for stock, suppliers, transactions & Reports

Hardware Interface

- Standard computer with 4GB RAM, 800GB HDD

Software Interface

- OS: Windows / Linux
- Database: MySQL / PostgreSQL
- Language: Java / Python / .NET (Dependency on implementation)

(5) Performance Req.

- The system must support at least 1000 stock items without performance lag
- Response time for stock search should be < 2s
- The system should handle simultaneous access by multiple users

(6) Security Req.

- Password-protected login system
- Role-based access (Admin has full control, staff has limited access)
- Database backup & restore facility

4. Non-functional Req.

- Reliability: System should have uptime of 99%
- Scalability: Should support more users / items when needed
- Usability: Easy to use GUI for non-tech staff
- Maintainability: Should allow easy updates & debugging

Estimated budget is £15,00,000.

4.3 Class Diagram

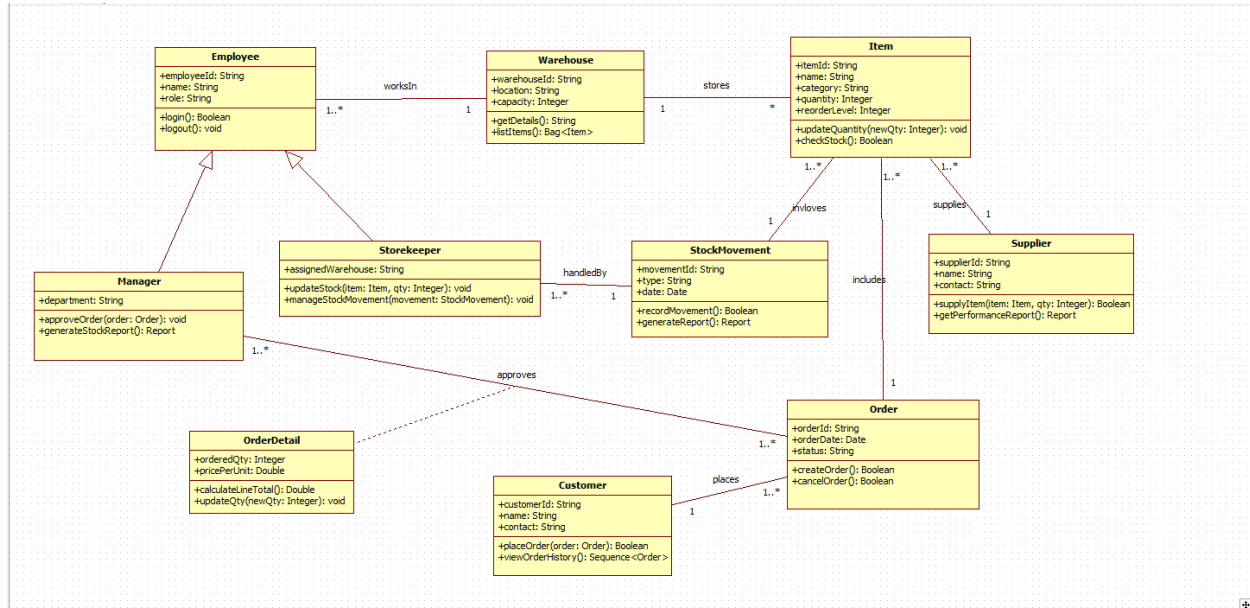


Fig 4.1

The simple class diagram shows the main classes involved in the Stock Maintenance System. The **Item** class stores details such as item name, category, quantity, and reorder level. The **Supplier** provides items to the system and is linked to the Item class. Customers place orders through the **Order** class, which contains information like order ID, date, and status. Each order includes details stored in the **OrderDetail** class.

The **Warehouse** class stores multiple items and maintains item lists. The **Employee** class represents workers in the system, and it has two specialized roles: **Manager**, who approves orders and generates reports, and **Storekeeper**, who updates stock and handles stock movements. Stock changes are recorded in the **StockMovement** class, which stores the type of movement and date.

Overall, the simple class diagram outlines how items, orders, employees, suppliers, and warehouses are connected to maintain stock efficiently.

4.4 Advanced Class Diagram

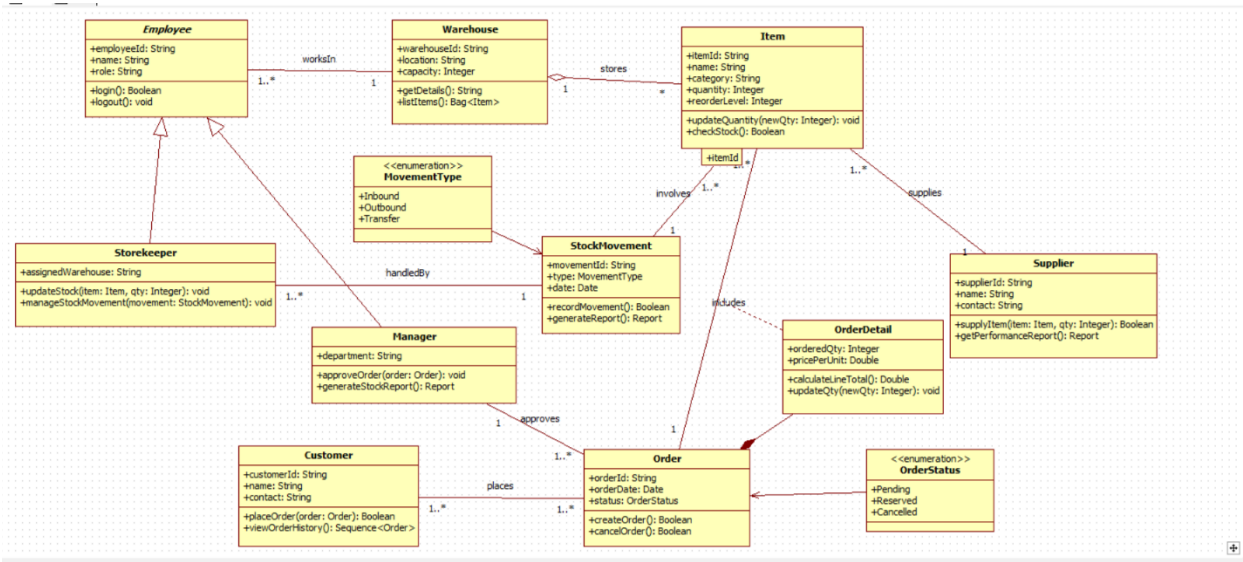


Fig 4.2

The advanced class diagram displays the structural model of the Stock Maintenance System.

- **Employee**, **Storekeeper**, and **Manager** manage stock operations and approvals.
- **Warehouse** stores multiple items.
- **Item** contains details like quantity, category, reorder level, and provides stock-checking methods.
- **Order** represents customer orders and is linked to **OrderDetail** for item quantity and pricing.
- **Supplier** provides items and maintains performance reports.
- **StockMovement** records inbound, outbound, and transfer movements using **MovementType** enumeration.

Relationships show how orders are placed, approved, supplied, and stored, forming a complete inventory management structure.

4.5 State Diagram

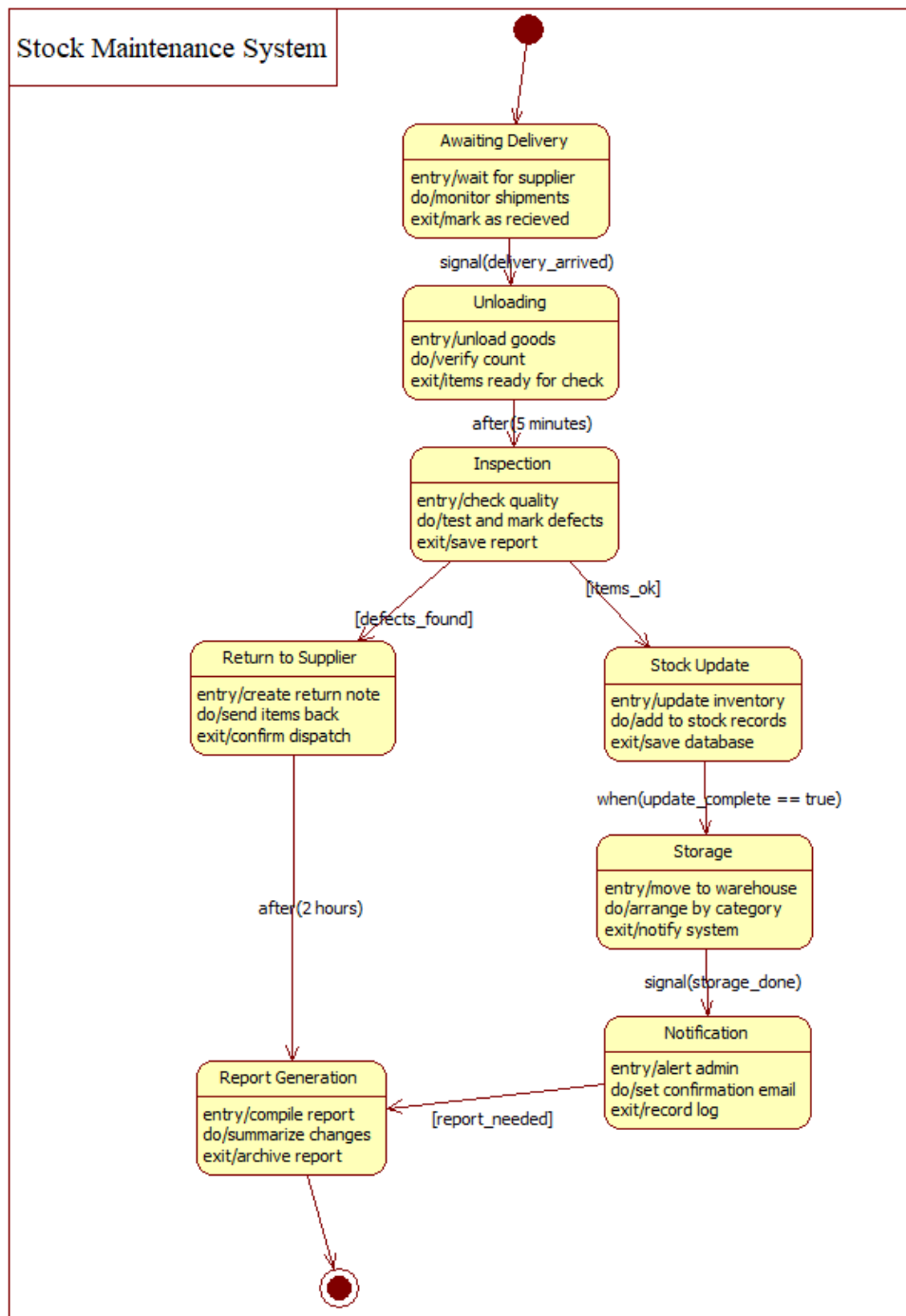


Fig 4.3

The state diagram models stock lifecycle. Items move through states like Awaiting Delivery, Unloading, Inspection, Returning to Supplier (if defects found), Stock Update, Storage, and Notification. Each state has entry, do, and exit actions such as verifying count, checking quality,

updating records, and notifying admin. The flow ensures that stock is properly received, validated, updated, and stored.

4.6 Advanced State Diagram

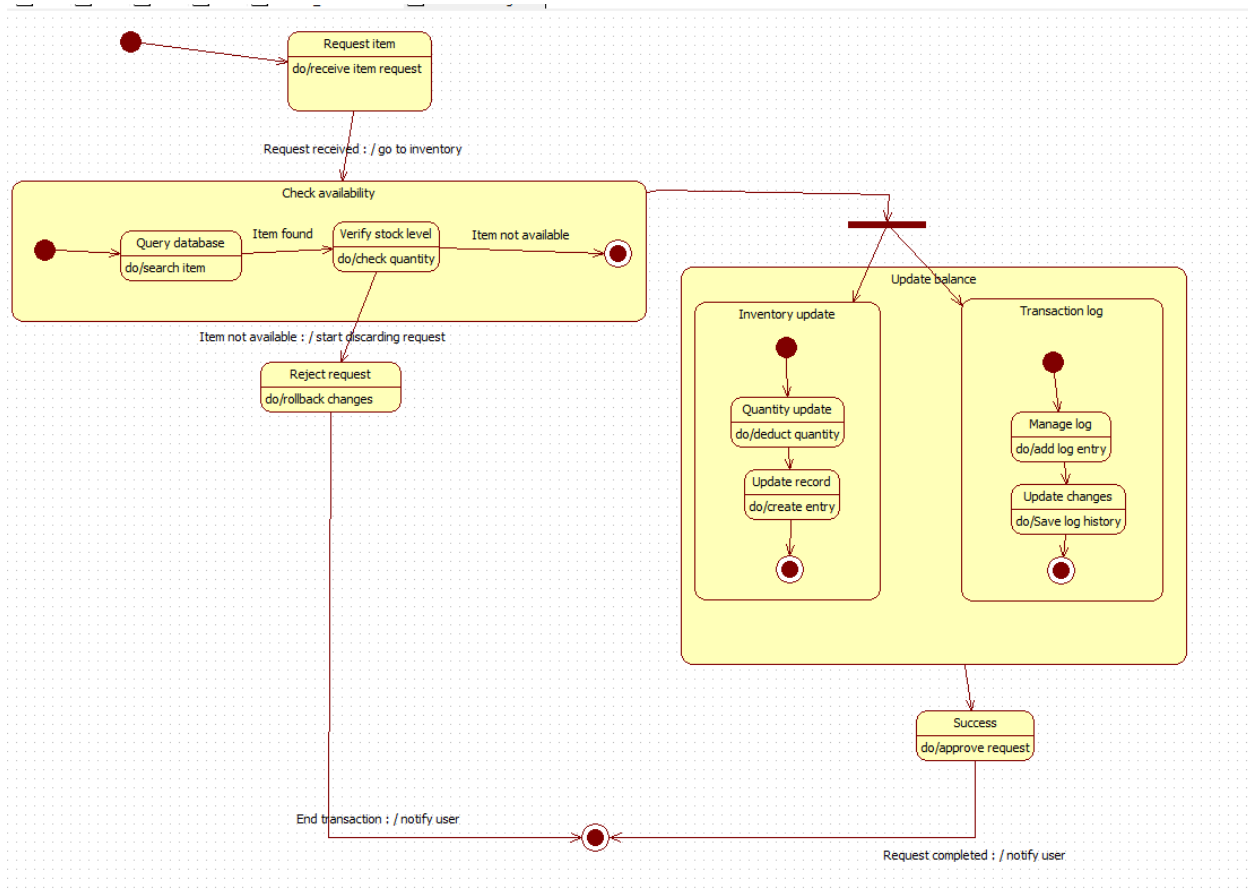


Fig 4.4

The advanced state diagram shows how stock items move through different stages during the maintenance process. The system begins in the **Awaiting Delivery** state, where it waits for items to arrive from the supplier. Once delivered, the items move to the **Unloading** state, where they are checked and counted.

Next, the items enter the **Inspection** state, where quality and defects are verified.

- If defects are found, the system transitions to **Return to Supplier**, where items are sent back.
- If the items are fine, they move to **Stock Update**, where inventory records are updated.

After updating, the items enter the **Storage** state, where they are arranged and stored properly. The system then moves to the **Notification** state to alert the admin and finally ends in **Report Generation**, where a summary report is prepared.

This diagram clearly represents the flow of stock from arrival to inspection, update, storage, and reporting.

4.7 Use Case Diagram

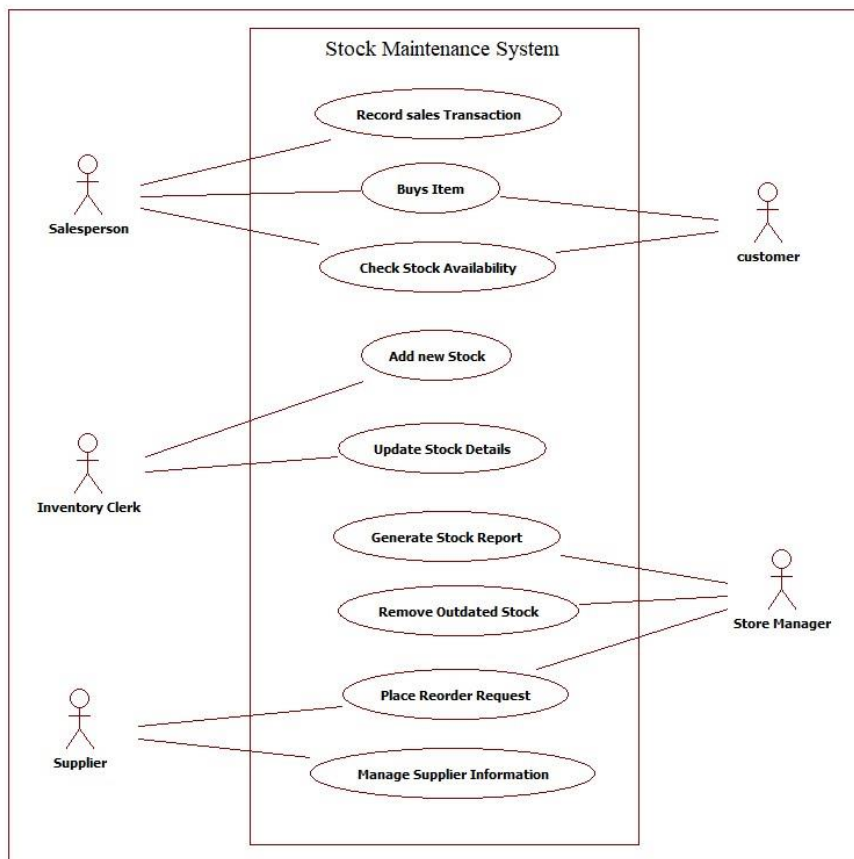


Fig 4.5

This version focuses on key roles: Salesperson records sales and checks item availability; Inventory Clerk adds and updates stock; Store Manager removes outdated stock, generates reports, and places reorder requests; Suppliers manage supplier information. Customers can purchase items and view availability.

Overall, this diagram highlights external interactions with the stock system.

4.8 Advanced Use Case Diagram

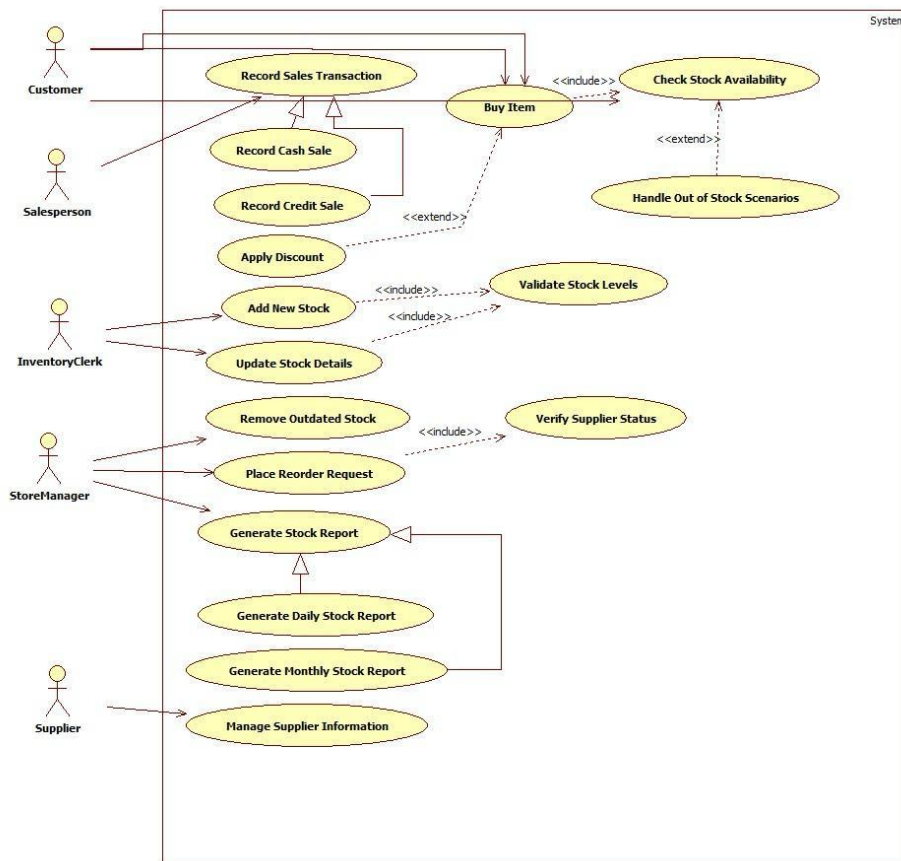


Fig 4.6

The advanced use case diagram shows all major actors and their interactions with the Stock Maintenance System. **Salespersons** buy and record sales transactions while checking stock availability. **Inventory Clerks** manage stock by adding new items, updating stock details, and removing outdated stock. **Store Managers** generate daily and monthly stock reports and place reorder requests when inventory is low. **Suppliers** provide stock and manage supplier information for replenishment.

Some use cases include or extend others—for example, validating stock levels is included in both adding and updating stock, and handling out-of-stock situations extends the “Buy Item” use case. Overall, the diagram captures how different users collaborate to maintain accurate inventory, ensure stock availability, and support efficient stock operations.

4.9 Sequence Diagram

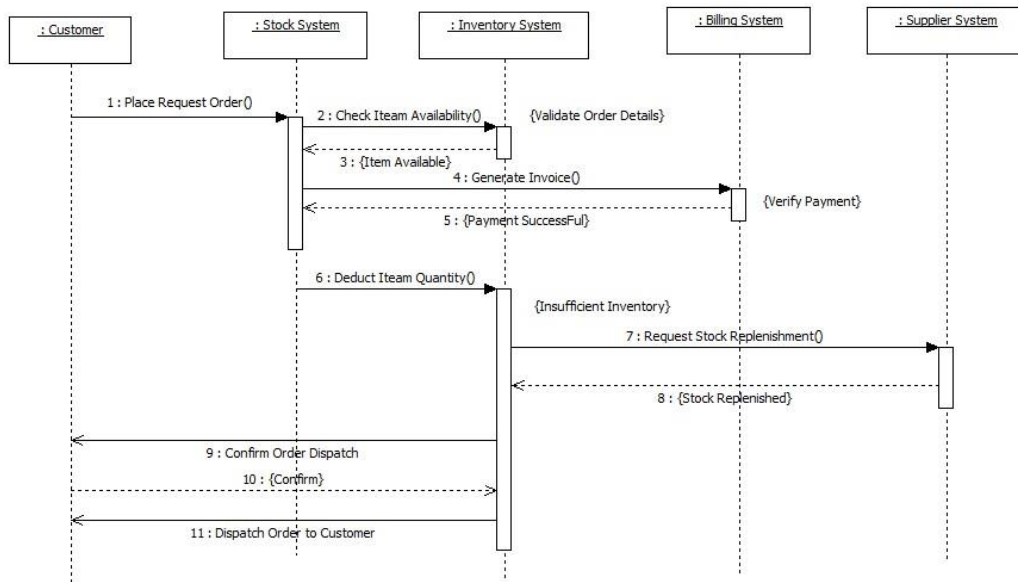


Fig 4.7

The sequence diagram shows how the Admin interacts with the Stock Management System to add or update stock. When stock is added, the system inserts the details into the inventory database and returns the confirmation. During stock update, the system deducts quantity, updates the database, and sends back the updated status. A report object is then created and used to generate final stock reports before being destroyed. This diagram highlights communication between Admin, Stock System, and Database.

4.10 Advanced Sequence Diagram

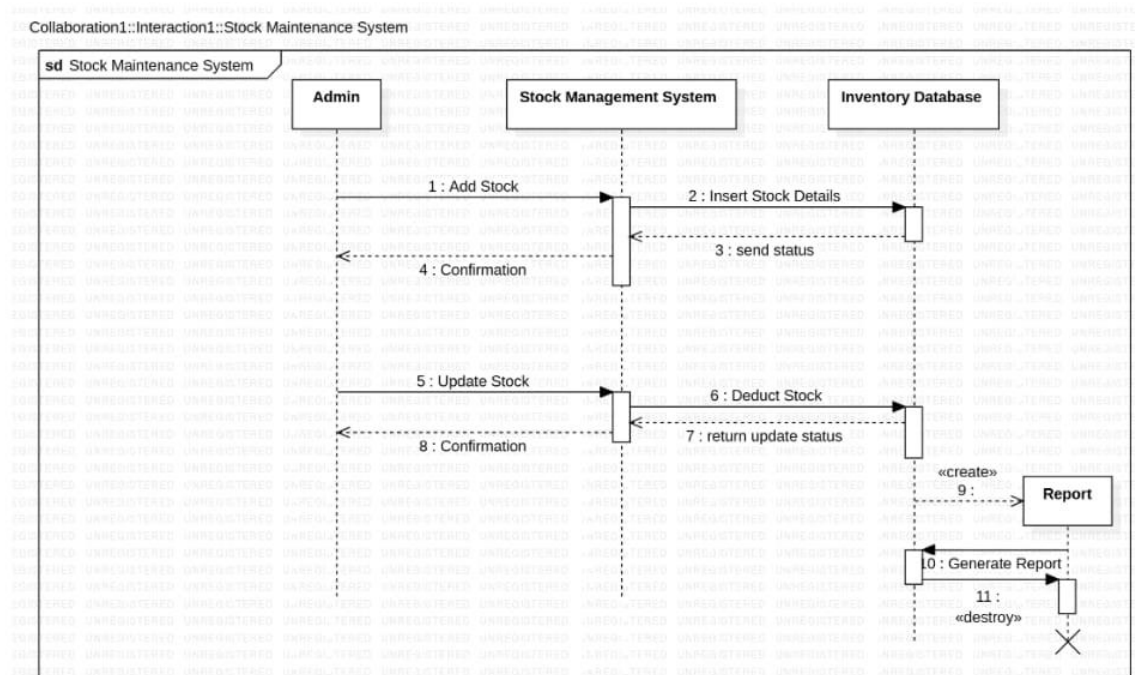


Fig 4.8

The sequence diagram illustrates how the Stock Maintenance System processes stock-related operations. First, the Admin sends an “Add Stock” request to the Stock Management System, which then inserts the new stock details into the Inventory Database. The database returns a status message, and the system sends a confirmation back to the Admin. Similarly, when the Admin sends an “Update Stock” request, the system forwards a stock deduction request to the database, receives the updated status, and confirms the update to the Admin. After these operations, a Report object is created, and the system generates a detailed stock report before destroying the report object. Overall, the diagram clearly shows the step-by-step communication flow between the Admin, Stock Management System, and Inventory Database during stock addition, update, and report generation.

4.11 Activity Diagram

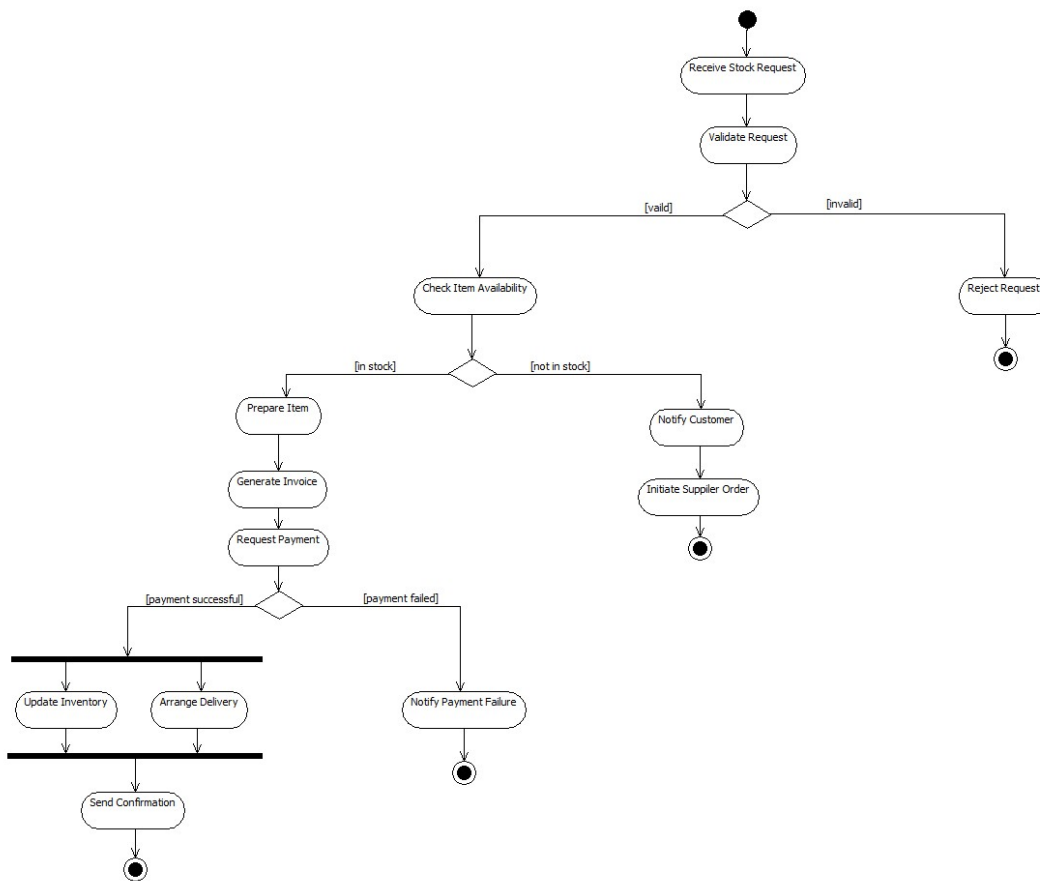


Fig 4.9

The activity diagram explains the workflow of a stock request process. The request is validated, item availability is checked, and based on availability, the system either prepares the item or initiates a supplier order. Next, invoice is generated, payment is requested, and depending on success or failure, the system updates inventory, arranges delivery, or notifies payment failure. The process ends after sending confirmation to the requester.

4.12 Advanced Activity Diagram

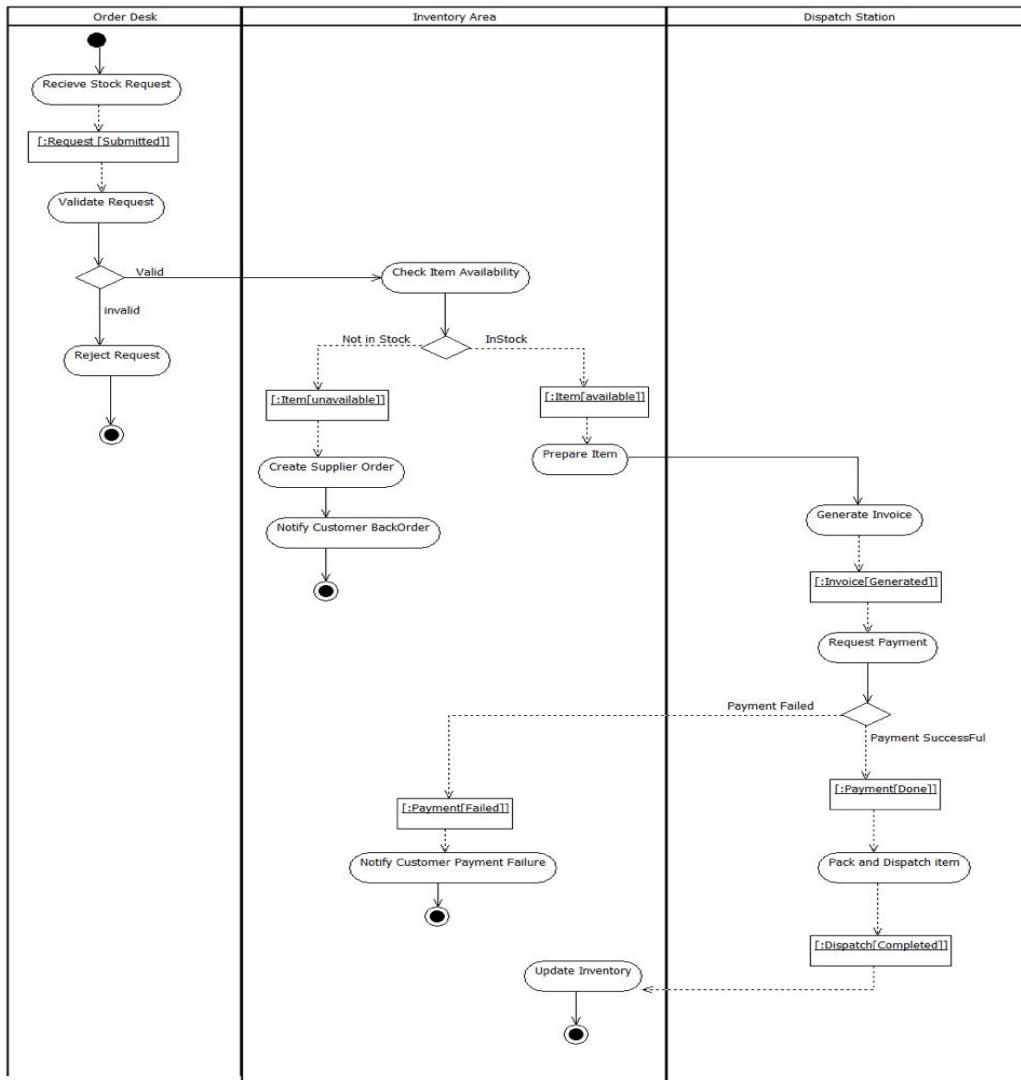


Fig 4.10

The activity diagram explains the workflow of a stock request process. The request is validated, item availability is checked, and based on availability, the system either prepares the item or initiates a supplier order. Next, invoice is generated, payment is requested, and depending on success or failure, the system updates inventory, arranges delivery, or notifies payment failure. The process ends after sending confirmation to the requester.

5. Passport Automation System

5.1 Problem Statement

The Passport Automation System aims to simplify and automate the entire passport application process, starting from submitting an application to the final issuance of the passport. The system allows applicants to apply online, schedule appointments, upload documents, and track their application status. Passport officers verify documents, conduct biometric checks, update status, and process approvals or rejections. The system also coordinates with the Police Department to perform background verification. By integrating all stakeholders—Applicant, Passport Office, Verification Officer, and Police Department—the system ensures faster processing, reduced manual work, secure document handling, and complete transparency for the applicant.

5.2 SRS-Software Requirements Specification

Passport Automate system

to design & implement a 'Passport Automate System' that simplifies & automates the process of passport application, renewal & tracking. Currently passport processing is largely manual, involving long queues, paperwork & inefficiencies that lead to delays, errors, fraud, & poor customer service. An automated system is needed to allow applicants to apply online, upload documents, schedule appointments, track application status & receive notifications.

SRS

1. Introduction

1.1 Purpose

The purpose of this SRS is to define the requirements for the Passport Automate System. The system will allow applicants to submit passport applications online, reduce paperwork, provide transparency, & assist passport office staff in processing & verifying applications efficiently.

1.2 Scope

The Passport Automate System will support online applications & renewal of passports. Allow applicants to upload documents digitally. Provide appointment scheduling for biometric & police verification. Enable real time application tracking & status notified.

Overview: Applicant submission, payment, status tracking.
 User Module: Applicant submission, payment, status tracking.
 Admin Module: Document verification, police verification, approval.
 Database Module: Store applicant details, application history.
 Notification Module: Sends updates via SMS/Email.

- (2) General Description
- Users: Applicants, passport officials, police verification officers, admin.
 - System Environment: Web-based system accessible via browser & mobile devices.
- (3) Functional Req
- User Registration & login: Users can register using personal details.
 - Secure login with username/password/OTP.
 - Apply for passport.
 - All online application form.
 - Upload req documents.
 - Pay application fee online.
 - Verification Process: Officials verify documents.
 - Police verification integrated.
 - Appointment scheduling: System auto-generates available slots.
 - User can reschedule appointments.
 - Status tracking & Notification: Applicants can track application progress.
 - Automatic notification via SMS/email.

- (4) Interface req
- User Interface:
 - Web & mobile-friendly UI.
 - Forms for application, fee.
 - Dashboard for users & admin.
 - Hardware Interface:
 - Biometric devices.
 - Secure servers for database storage.
 - Software Interface:
 - Payment gateway.
 - SMS/email server/provider.
 - National ID database.
- (5) Performance Req
- System should support 10,000+ concurrent users.
 - Response time must be less than 2s for user action.
 - Database should handle millions of records efficiently.
- (6) Design Constraints
- Must comply with gov security & privacy policies.
 - Must follow standard encryption protocols for data transmission.
 - Support multi-lingual functionality.
 - Operate on standard web browsers & mobile devices.
- (7) Non-functional Attributes
- Security: End-to-end encryption, role-based access.
 - Reliability: Data backup & recovery mechanisms.
 - Maintainability: Modular design for updates.
 - Scalability: Cloud support for handling growth.
 - Usability: Simple & user-friendly UI for citizens.
- (8) Preliminary Schedule & Budget
- | | |
|--|--------------------------------|
| 1. RA - 1 month | Testing & Deployment - 1 month |
| 2. System design - 1 month | Dev & training - 1 month |
| 3. Implementation, Module Dev - 2 months | |

Budget estimate

Software dev - \$50,000
 Hardware & servers - \$30,000
 Security & Compliance - \$20,000
 Maintenance (1st yr) - \$15,000
 Total Estimated Budget :- \$115,000

Qw
 28/8

5.3 Class Diagram

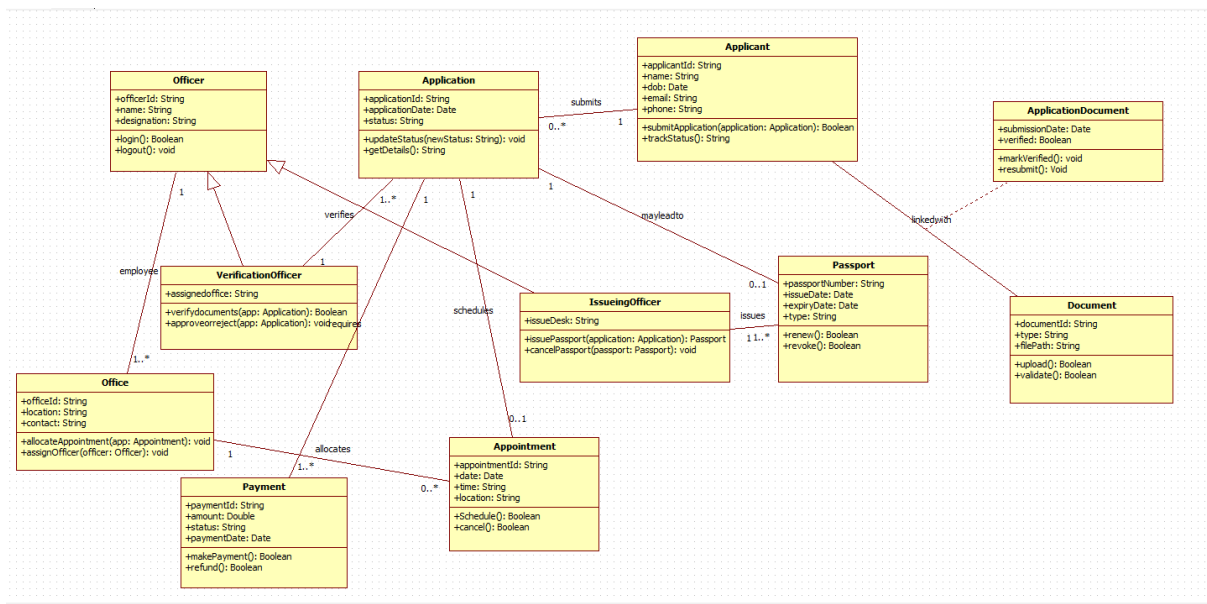


Fig 5.1

The class diagrams show the structure of the Passport Automation System. Important classes include Applicant, Application, Passport, Officer, Appointment, Payment, and Document. Each class has attributes and functions, and the relationships show how they are connected—for example, applicants submit applications, officers verify them, and documents belong to applications.

5.4 Advanced Class Diagram

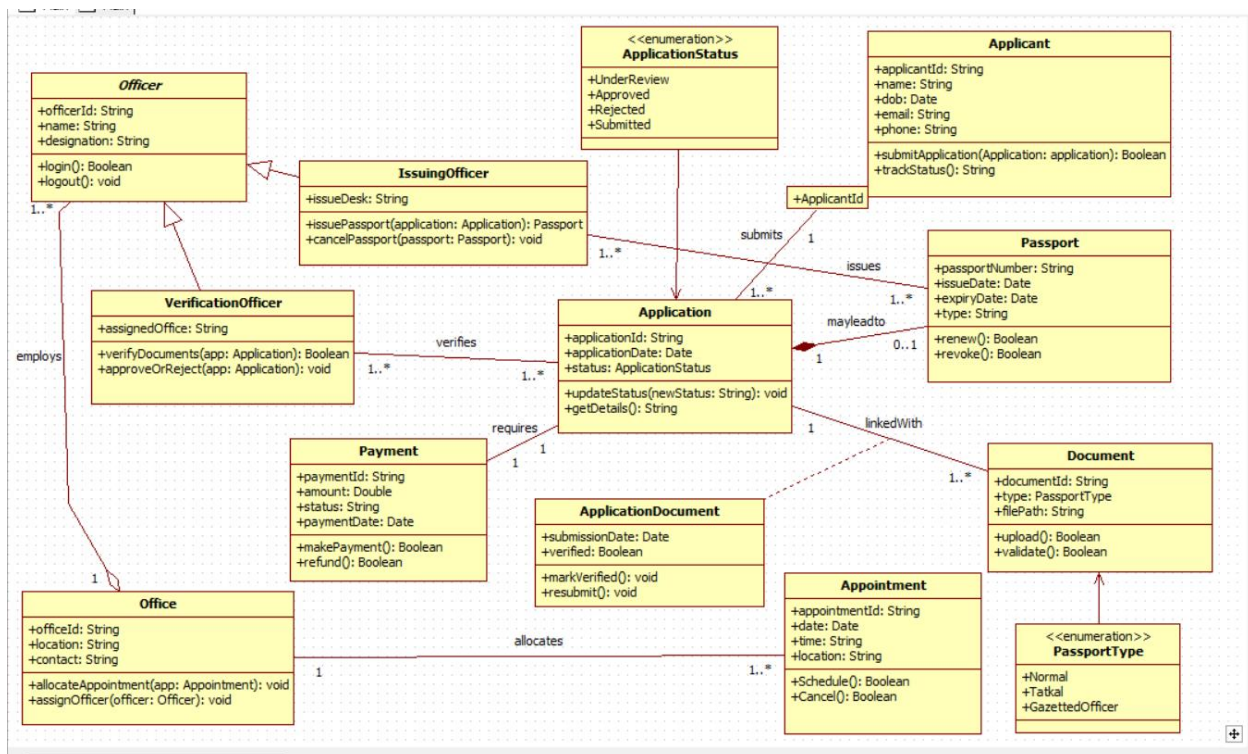


Fig 5.2

The advanced class diagrams define the complete object-oriented architecture of the system. They capture relationships such as associations, aggregations, compositions, multiplicities, and enumerations. The Application class links to Applicant, Documents, Payment, Appointment, and Passport, forming the core of the system. Specialized roles like VerificationOfficer and IssuingOfficer extend the Officer class through inheritance. Enumerations such as ApplicationStatus and PassportType provide controlled value sets. Method signatures describe behavior like verifyDocuments(), approveOrReject(), allocateAppointment(), and issuePassport(). Together, the diagrams establish the system's static design blueprint.

5.5 State Diagram

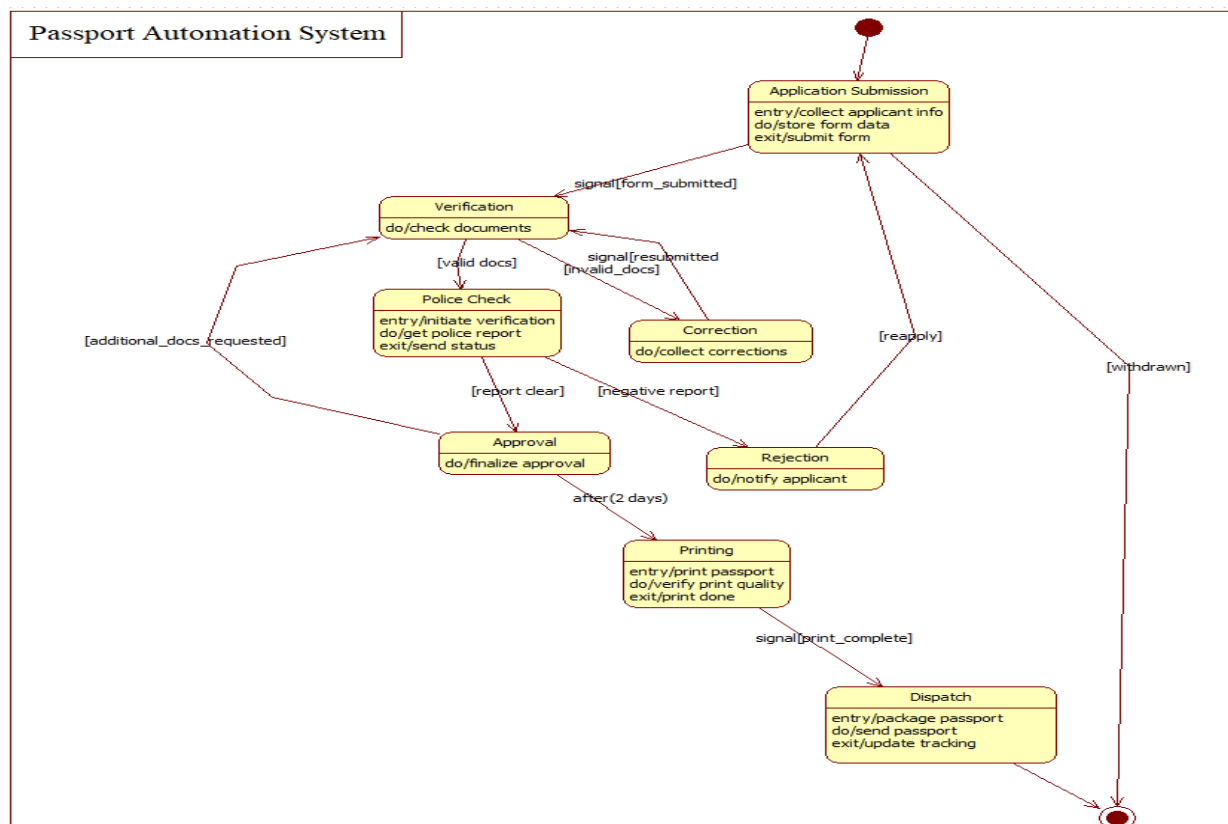


Fig 5.3

The state diagrams show how an application moves from one stage to another. It starts at "Application Submitted", goes through document verification, biometric verification, police verification, approval, printing, and finally passport dispatch. Each state represents the current position of the application in the process.

5.6 Advanced State Diagram

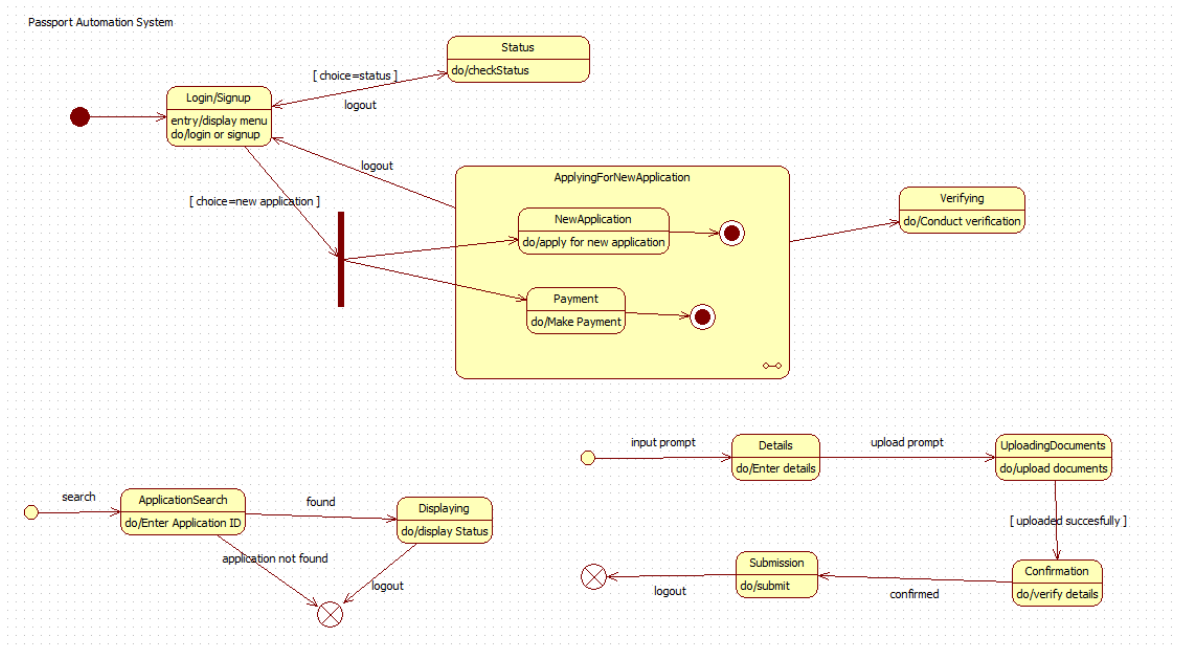


Fig 5.4

The advanced state machine diagrams detail the dynamic lifecycle of a passport application. Each state contains entry/do/exit actions that specify internal operations such as storing form data, validating documents, initiating police checks, performing corrections, or printing passports. Transition triggers like form_submitted, invalid_docs, report_clear, and print_complete control the progression between states. Special conditions handle resubmission, withdrawal, and rejection. The model demonstrates reactive system behavior based on events, delays, and signals, providing a complete state-based representation of the application process.

5.7 Use Case Diagram

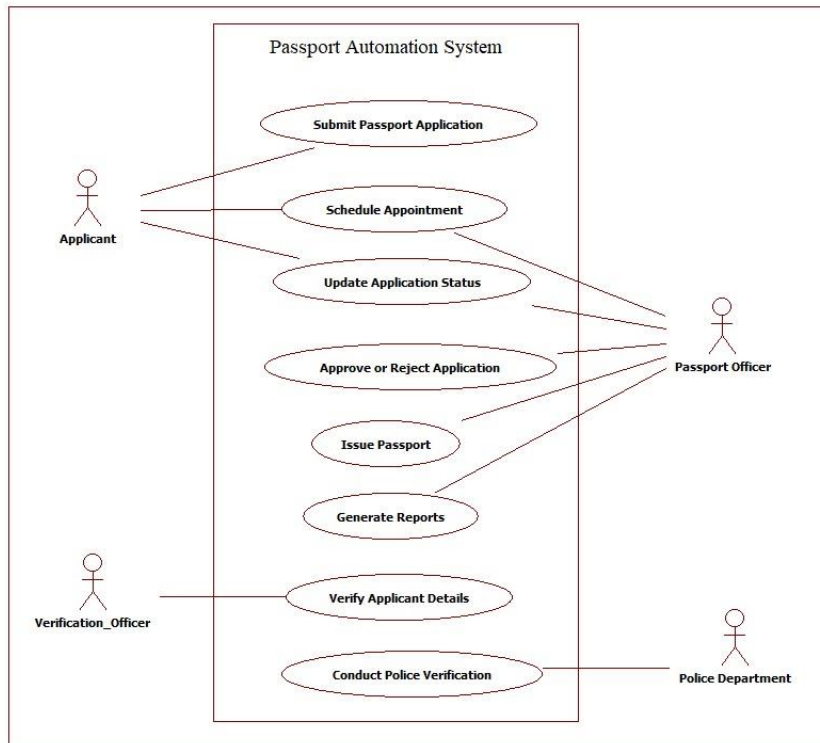


Fig 5.5

The use case diagrams show the different actions that users can perform in the Passport Automation System. Applicants can submit applications, schedule or reschedule appointments, upload documents, and track their status. Passport officers verify the application, approve or reject it, and issue passports. Verification officers check documents and biometrics, while the police department performs background verification. The diagram simply shows how each actor interacts with the system.

5.8 Advanced Use Case Diagram

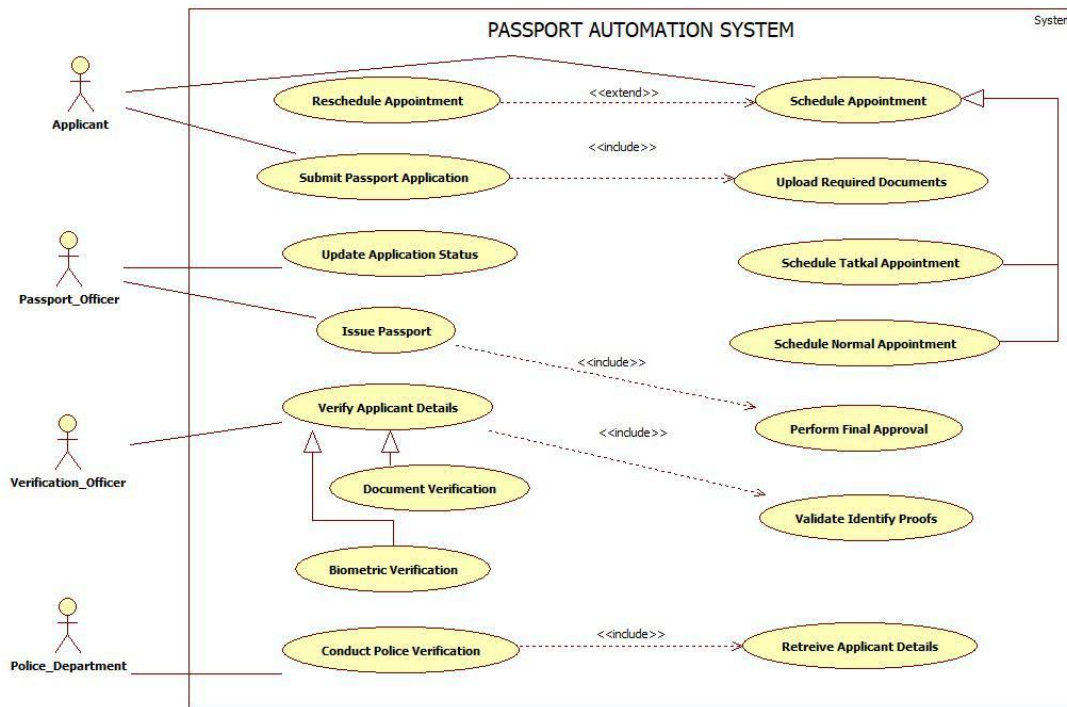


Fig 5.6

The advanced use case diagrams present detailed functional interactions between system actors and the passport automation workflow. They illustrate mandatory processes such as document submission, biometric enrollment, identity verification, police background check, and final approval. <<include>> and <<extend>> relationships highlight modularity—for example, biometric verification and police verification are mandatory sub-processes under applicant details verification. The diagrams also differentiate between normal and Tatkal appointments, extended rescheduling flows, and system-driven processes such as retrieving applicant details and validating identity proofs. Overall, they model the complete functional scope of the passport system.

5.9 SEQUENCE DIAGRAMS

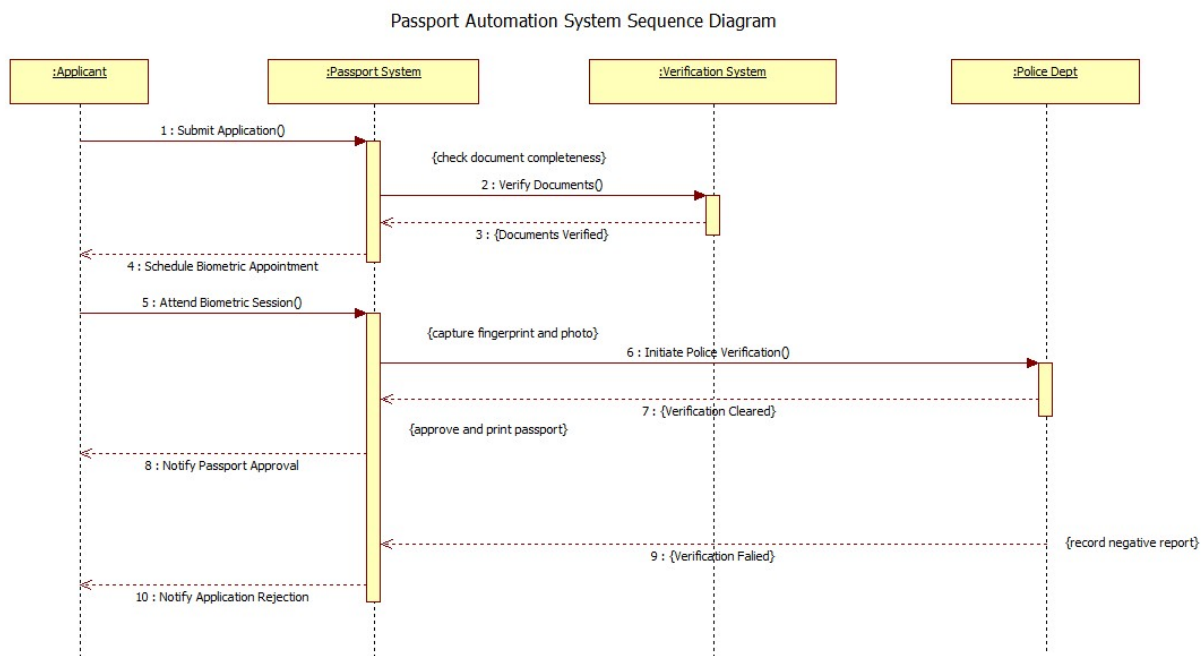


Fig 5.7

The sequence diagrams show how the passport application process happens step-by-step. First, the applicant submits the application, then the passport system checks documents, schedules biometrics, and sends the request to verification and police departments. After verification is completed, the system approves or rejects the application, and the applicant is notified.

5.10 Advanced SEQUENCE DIAGRAMS

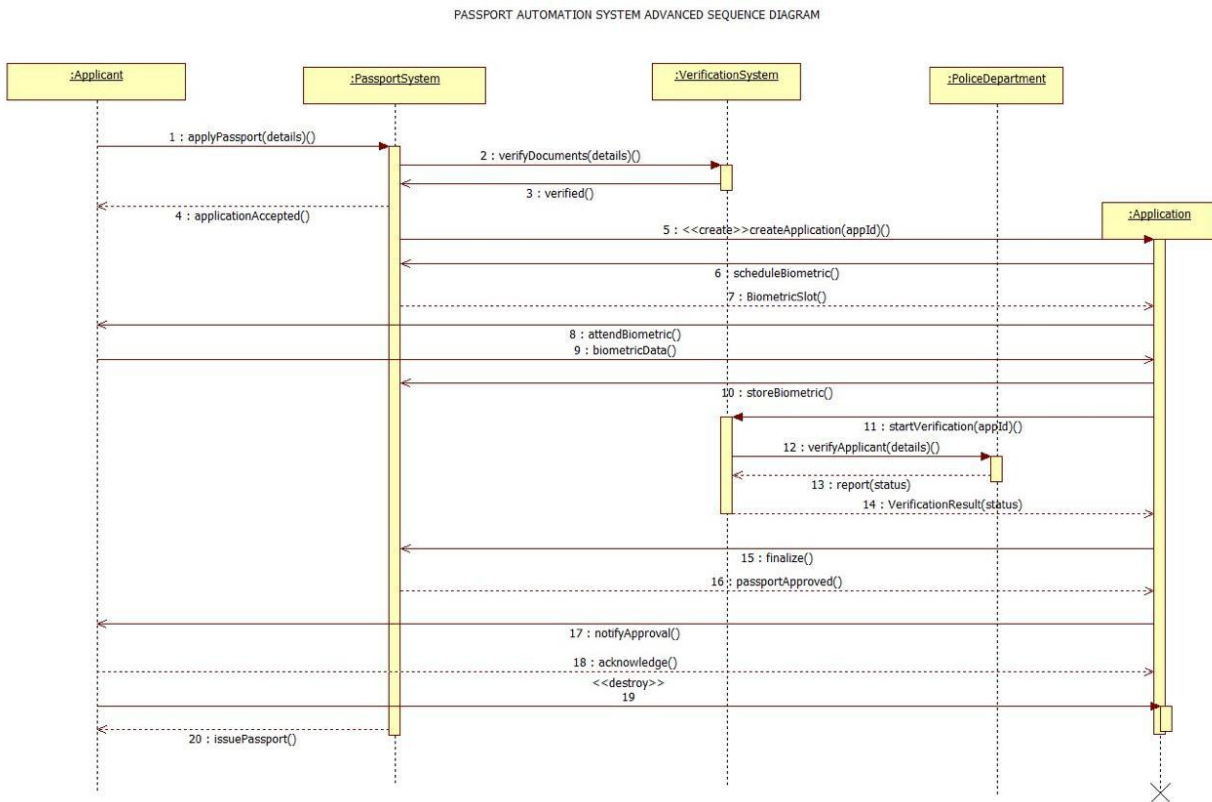


Fig 5.8

The advanced sequence diagrams illustrate the chronological interaction between system components: Applicant, Passport System, Verification System, and Police Department. They capture message passing such as document verification, biometric scheduling, biometric storage, police verification requests, status reporting, and application finalization. The diagrams also show object creation (like Application object instantiation), conditional flows, and system

notifications. This demonstrates how control moves across lifelines, ensuring process synchronization from submission to passport issuance and closure.

5.11 ACTIVITY DIAGRAMS

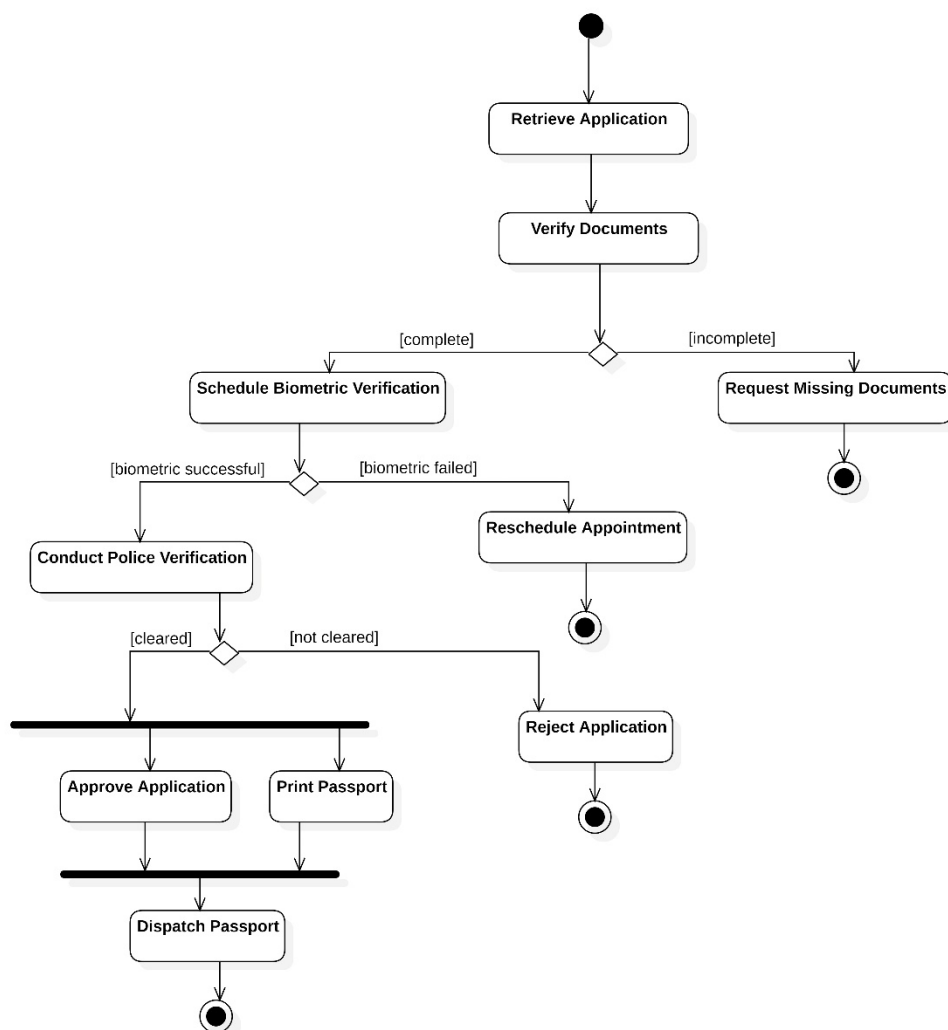


Fig 5.9

The activity diagrams show the overall workflow of the passport process. It starts with the applicant submitting the application. The office then verifies the documents and schedules biometrics. After biometrics, the police department performs verification. Based on the results, the application is either approved, printed, and dispatched or rejected.

5.12 Advanced ACTIVITY DIAGRAMS

PASSPORT AUTOMATION SYSTEM ADVANCED ACTIVITY DIAGRAM

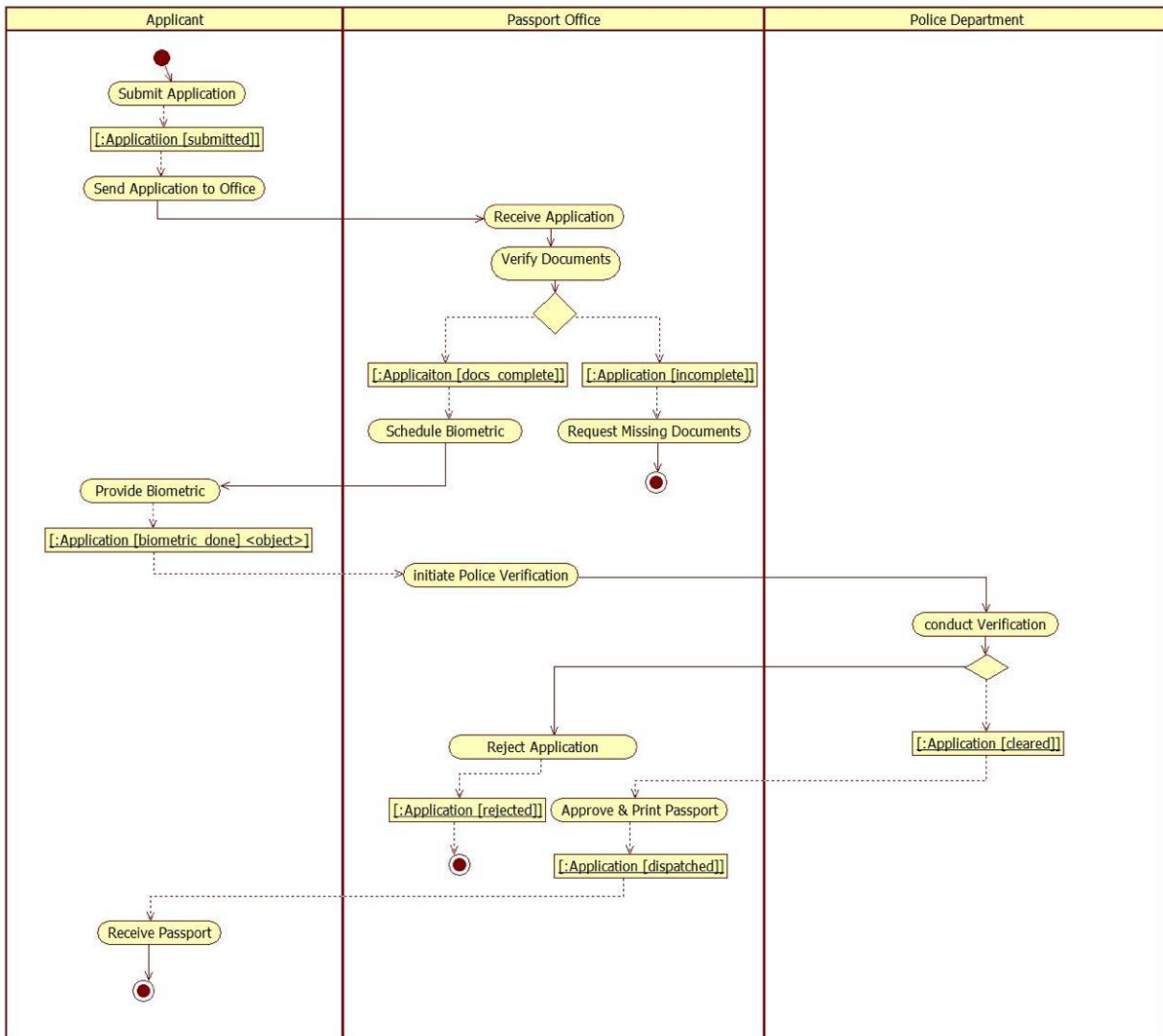


Fig 5.10

The advanced activity diagrams highlight parallel actions, decisions, and cross-department workflows through swimlanes. They illustrate branching conditions such as incomplete documents, biometric failure, or negative police verification. Synchronization bars show concurrent actions like approval and printing before dispatch. The diagrams emphasize event-driven transitions (like applicationSubmitted, biometric_done, application_cleared) and exception flows involving missing documents or rejection. This provides a complete behavioral representation of the system's operational process.